
Department of Computer Engineering

1403108: Operating Systems

MODULE- II: File Systems and Device Management

Prerequisites Courses: Computer Fundamentals

MODULE II -File Systems and Device Management

Module Contents:

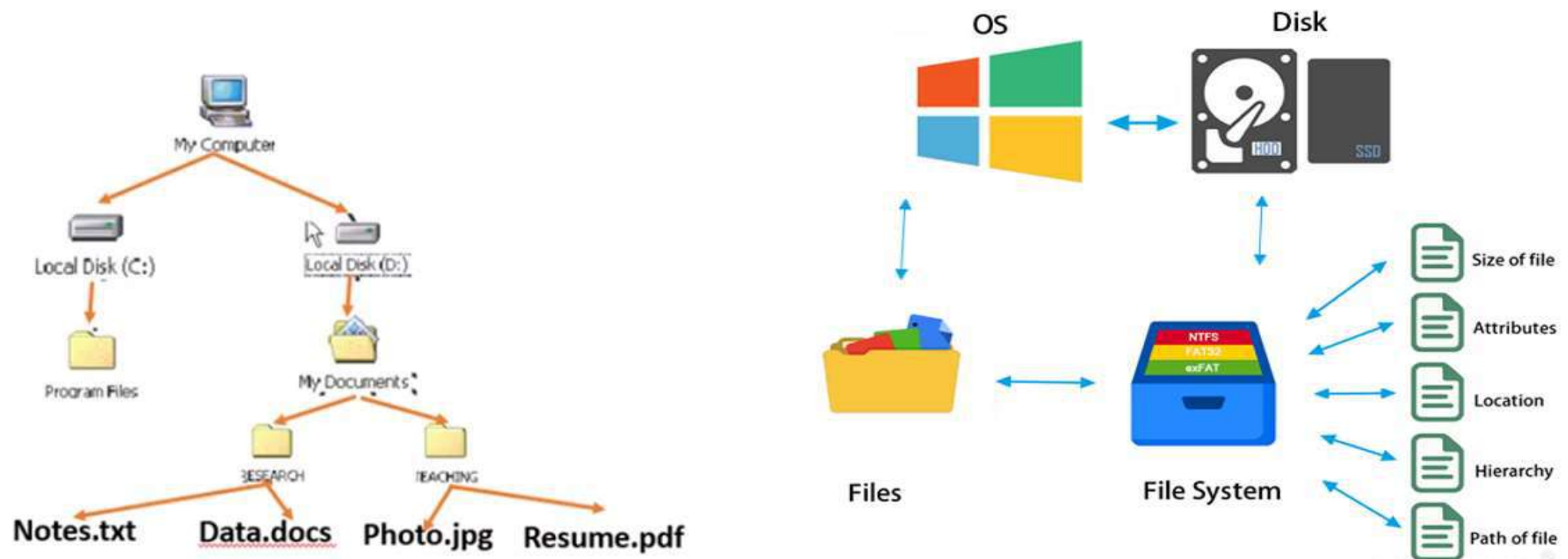
- **File Systems:** Overview, File Organization and Access, File Directories, File Sharing, Record Blocking, Secondary Storage Management, File System Security, UNIX File System, Linux Virtual File System, Windows File System, Case studies of advanced file systems (ext4, ZFS, Btrfs).
- **Device Management:** I/O Devices, Organization of the I/O Function, Operating System Design Issues, I/O Buffering, I/O hub, Disk Scheduling Policies: FIFO, PRIORITY, SSTF, SCAN, C-SCAN, Linux I/O, Windows I/O.

File Systems: Overview, File Organization and Access

About File & Directory in OS

➤ **Files** is a collection of co-related information that is recorded in some format .

- Stored on various storage mediums such as flash drives, hard disk drives (HDD), magnetic tapes, optical disks, and tapes etc.



File Systems: Overview, File Organization and Access

Why Disk Management is Important

Most computer systems use secondary storage devices (like hard disks, SSDs, tapes, optical media, and flash drives) to store programs and data at low cost and with non-volatile storage.

Data is stored in the form of files.

The operating system (OS) manages file storage by allocating disk space as needed. Files are not always stored in one continuous block, large files may be fragmented into parts stored in different disk locations, especially when space is limited.

The OS keeps track of where each file (and its fragments) is located, often handling thousands of such entries. It ensures:

- Files can be quickly located for read/write operations
- Safe and reliable access to stored data
- Efficient management of access times

File Systems: Overview, File Organization and Access

SSDs use flash memory for fast, silent, and durable data storage, ideal for operating systems and applications, while HDDs use magnetic spinning disks for high-capacity, cost-effective, long-term storage. **SSDs are faster and more durable, whereas HDDs offer lower costs per gigabyte.**

It ensures:

- Efficient use of storage space, Safe and reliable read/write operations
 - Faster access times
- disk management techniques used in operating systems include:



- **Partitioning:** Divides a physical disk into multiple logical partitions, each acting as a separate storage device for better organization.
- **Formatting:** Prepares a disk by creating a file system; erases all existing data.
- **File System Management:** Manages file systems (e.g., FAT, NTFS, ext4) to store and access data efficiently.
- **Disk Space Allocation:** Allocates space for files using methods like contiguous, linked, or indexed allocation.
- **Disk Defragmentation:** Rearranges scattered data blocks to improve performance.

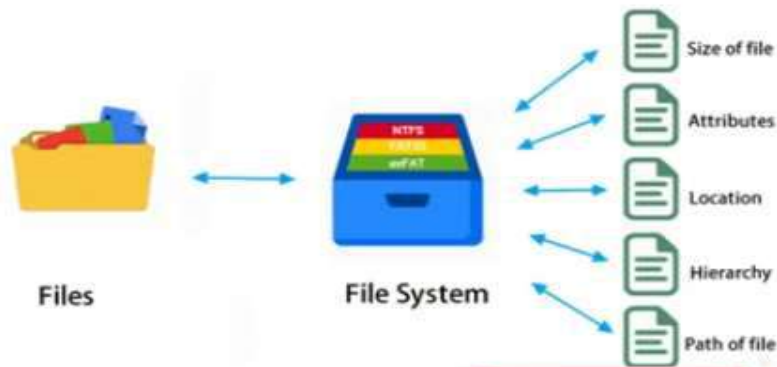
File Systems: Overview, File Organization and Access

About File System

- It is a method used by an OS to organize and manage files on a storage device.

Popular File Systems:

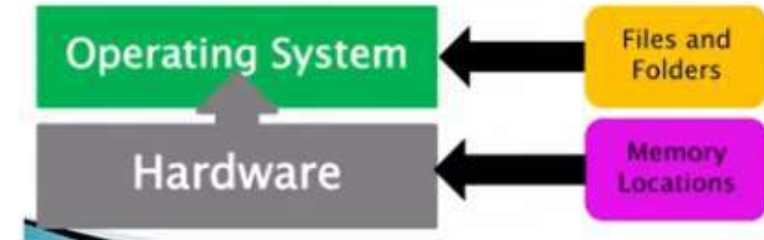
- FAT (File Allocation Table):** An older file system used by older versions of Windows.
- NTFS (New Technology File System):** Used by Windows. Supports features such as file and folder permissions, compression and encryption.
- ext (Extended File System):** Commonly used on Linux and Unix-based operating systems.
- HFS (Hierarchical File System):** A file system used by macOS.
- APFS (Apple File System):** Introduced by Apple for their Macs and iOS devices.



File Systems: Overview, File Organization and Access

File Management

- Files are used for long-term storage.
- Files are used for both input and output.
- Every operating system provides a file management service.
- This file management service can also be treated as an abstraction as it hides the information about the disks from the user.



➤ The OS provide system call for file management

1. File creation
2. File deletion
3. Read and Write operations

File Systems: Overview, File Organization and Access

Functions of File Management

1. **File Creation** -> notes.txt or resume.pdf
2. **File Deletion** -> Deleting old_video.mp4
3. **File Reading**-> Opening a PDF or viewing a page
4. **File Writing / Modification** -> Editing a Word document
5. **File Renaming** -> Changing abc.txt → notes.txt
6. **File Organization and Storage Management** -> contiguous, linked, indexed
7. **File Access Control** -> Permissions like Read, Write & Execute
8. **File Security and Protection** -> Password, Encryption, Authentication
9. **Directory Management** -> Documents → College → Notes.
10. **File Backup and Recovery** -> File History, Recycle Bin.

File Systems: Overview, File Organization and Access

File Operations

Sr. No.	Operation	Meaning	Example
1	Create	Make new file	create report.txt
2	Open	Load file to read/write	Double-clicking a file
3	Read	Read contents	OS reads bytes from file
4	Write	Add/modify data	Typing in Notepad
5	Append	Add data at end	Adding new line to log file
6	Close	Finish using file	Clicking Save & Close
7	Delete	Remove file	Delete song.mp3
8	Rename	Change name	abc.txt → notes.txt

File Systems: File Access and File access methods

File Access Methods

- File access methods are techniques used by an OS to read and write data in files.
- They define how information is organized, retrieved and modified.
- Choosing the right method is important for performance and data management.

➤ File Access Methods

Sequential Access

Direct Access

Indexed Access

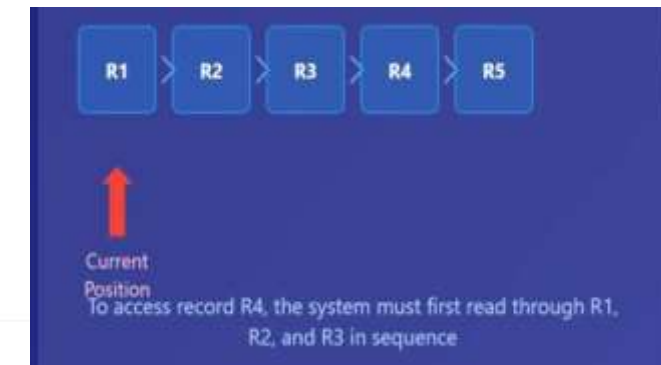
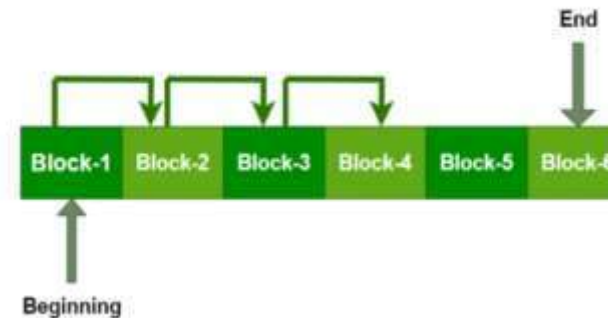


1. Sequential File Access

- Data is accessed from one record right after another record in an order.
- Read command, it moves ahead pointer by one.
- Write command, it will allocate memory and move pointer to the end of the file.
- ✓ Easy to implement but slow for searching specific records.
- ✓ Inserting/updating in the middle is difficult.

➤ Used in: Text files, Log files, Tape drives

📖 Reading a book page by page
🎬 Playing a movie from start to end



File Systems: File Access and File access methods

Sequential Access - Applications & Use Cases

Sequential access is ideal for scenarios where data needs to be processed in order



Batch Processing

Processing large volumes of data in batches where each record needs to be handled in sequence.

```
// Batch processing example
```



Log Files

System logs and transaction logs where entries are appended sequentially and typically read in chronological order.

```
// Reading a log file
```



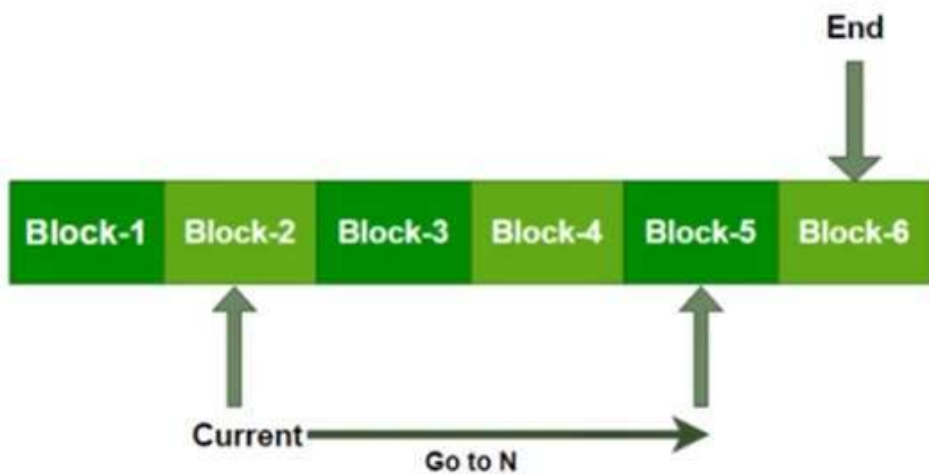
Tape Storage

Magnetic tape backups and archival storage where data is written and read sequentially by physical necessity.

```
// Tape backup pseudocode
```

2. Direct Access Method

- It allows data to be read or written directly at any block or record, using its address (block no).
- There is no need of traversing all the blocks present before it.
- ✓ Complex implementation which requiring sophisticated algorithms and data structures.
- ✓ It require additional storage for maintaining data location info. (such as pointers or add. Tables)
- **Used in:** Databases, Large binary files, Hard disks, SSD



🎵 Clicking directly on your favorite song in a playlist
🎬 Jumping to the 10th minute of a video

In a direct access file system, **a hash function is used to transform a record's unique key into a physical storage address (or disk block location)**, enabling rapid, near-constant-time data retrieval ($O(1)$ on average).

File Systems: File Access and File access methods

Direct Access Using Hashing

Hashing transforms a key into a direct address for efficient record retrieval



Hash Table

0	Johnson Data
1	Williams Data
2	Brown Data
7	Smith Data
9	Davis Data

Hashing calculates the storage address directly from the record key. This enables $O(1)$ access time in ideal cases, making it extremely efficient for direct access methods. However, collision handling is required when different keys hash to the same address.

Direct Access - Applications & Use Cases

Direct access methods excel in scenarios requiring immediate retrieval of specific records



Database Systems

Relational databases use direct access to quickly retrieve records based on primary keys or indexed fields.

```
// SQL database lookup
```



Hash Tables & Maps

In-memory data structures like hash maps provide $O(1)$ access time for key-value lookups using direct addressing.

```
// Hash map implementation
```



File Systems

Modern file systems use direct access to quickly locate and retrieve files based on their inode or file allocation table entries.

```
// File system direct access
```

3. Index Sequential Method

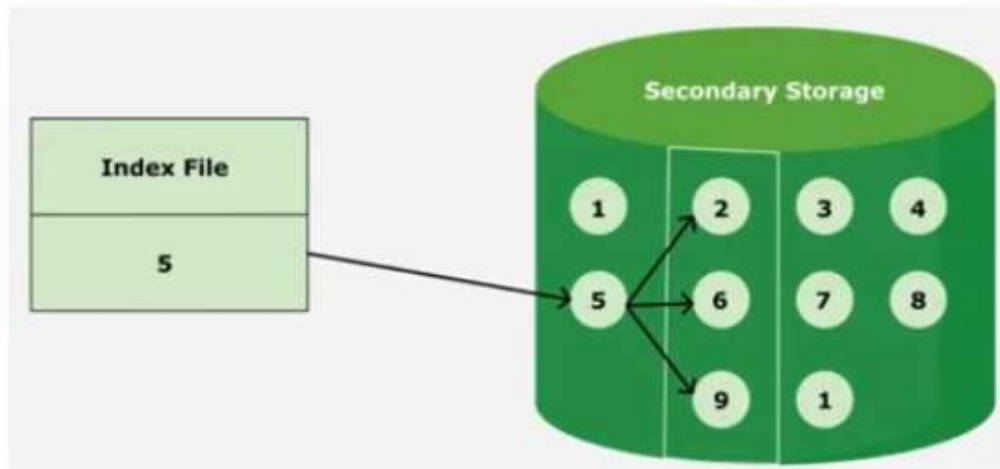
- These methods construct an index for the file.
- To find a record in file, we first search index & then by help of pointer we access file directly.
- ✓ Fast searching & reduce access time.
- ✓ Both data and index must be updated during insertions, deletions, or modifications.
- **Used in:** Databases system, Large record-based files.



Using index of a book (page number given)



Dictionary (first letter helps find word)



File Systems: File Access and File access methods

Index Table

Adams	Addr 2
Johnson	Addr 0
Smith	Addr 4
Williams	Addr 1



The index provides a mapping between keys and record addresses, allowing efficient lookup without scanning the entire file

B-Tree Index Structure

B-Trees are balanced tree data structures optimized for systems that read and write large blocks of data



⚡ B-Trees maintain balance automatically, ensuring consistent performance regardless of data distribution

📁 Searching a B-Tree with millions of records requires only a few disk accesses, making it highly efficient

🔄 B-Trees automatically reorganize when records are added or deleted to maintain optimal performance

File Systems: File Access and File access methods

Comparison of Access Methods

Evaluating sequential, direct, and indexed access methods across key performance factors

Criteria	Sequential Access	Direct Access	Indexed Access
Access Speed	Poor $O(n)$ - Must traverse records sequentially	Excellent $O(1)$ - Immediate access with address	Good $O(\log n)$ - Fast lookup via index
Storage Overhead	Excellent No additional storage required	Good Minimal overhead for address calculation	Average Additional space for index structures
Implementation Complexity	Simple Straightforward to implement	Moderate Requires address calculation logic	Complex Requires index maintenance
Update Efficiency	Poor May require rewriting entire file	Good Direct update if size unchanged	Average Must update both data and index

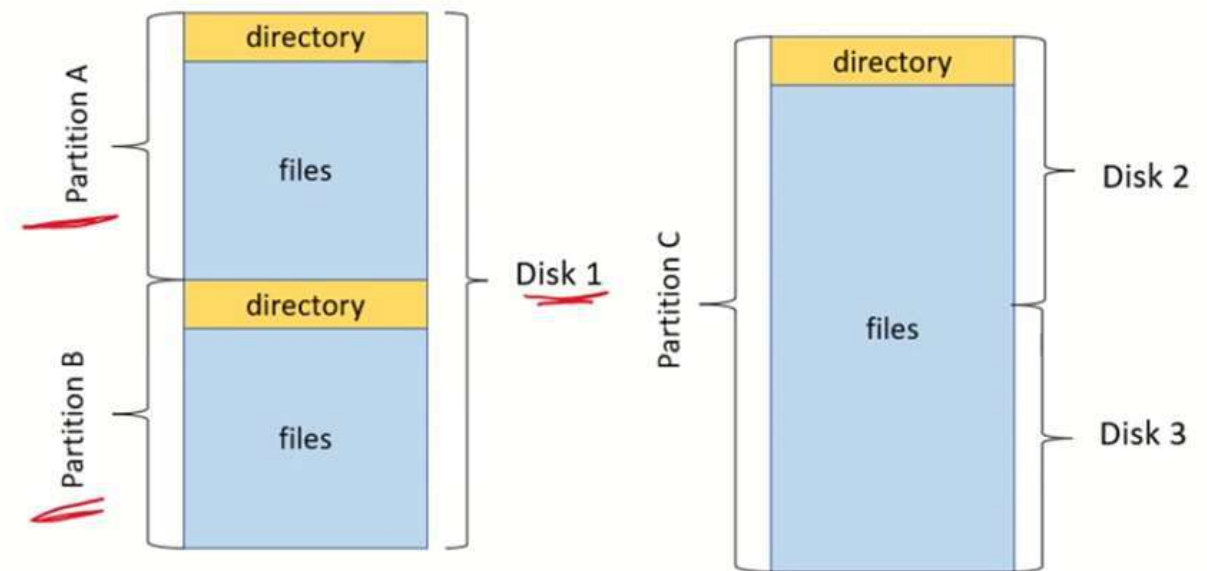
Choosing the Right Access Method

The optimal access method depends on specific application requirements. Sequential access works well for batch processing and complete file scans. Direct access excels when record addresses are known or can be calculated. Indexed access provides the best balance for applications requiring flexible queries on large datasets where keys are used for lookups.

File Systems: File Directories

Disk Structure

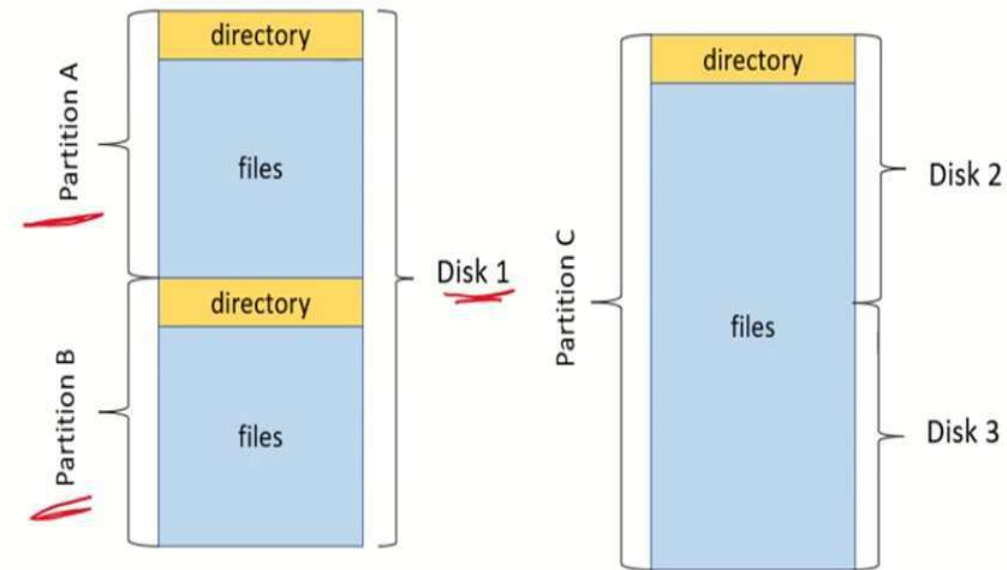
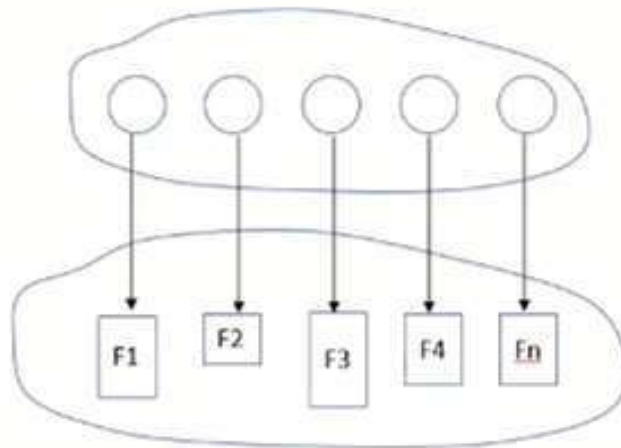
- Disk - subdivided into **partitions**
- Disk or partition
 - Can be used **raw** – without a file system
 - Or **formatted** with a file system
- Entity containing file system known as a **volume**
 - Volume tracks file system's info in **device directory** or **volume table of contents**



File Systems: File Directories

Directory Structure

- A collection of nodes containing information about all files
- Both the directory structure and the files reside on disk



File Systems: File Directories

Directory Operations

Sr. No.	Operation	Meaning	Example
1	Create Directory	Make new folder	<code>mkdir</code> College
2	Delete Directory	Remove folder	<code>rmdir</code> oldFolder
3	Open Directory	Access contents	Opening "Documents" folder
4	List Directory	Show files inside	<code>ls</code> (Linux), <code>dir</code> (Windows)
5	Rename Directory	Change name	College → BTechNotes
6	Move Directory	Shift to another path	Move Pics folder into Documents
7	Traverse Directory	Move through directories	Opening nested folders

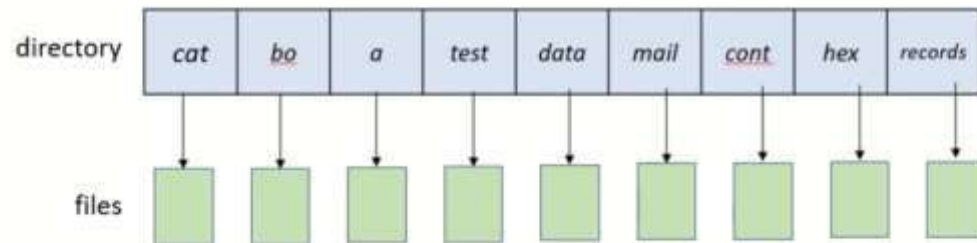
Operations Performed on Directory

- **Search** for file: Find entry for a particular file
- **Create** file: New files created and added to directory
- **Delete** file: When file no longer needed, remove it from directory
 - Leaves a hole in directory structure
 - File system may have to defragment directory structure
- **List** directory: list files and contents of directory entry for each file
- **Rename** file: must be able to change name when contents or use of file changes
 - May also allow its position within directory structure to be changed
- **Traverse** file system: access every directory and every file within a directory structure

File Systems: Overview, File Organization and Access

Single-Level Directory

- All files, for all users, contained in same directory
- Easy to support and understand

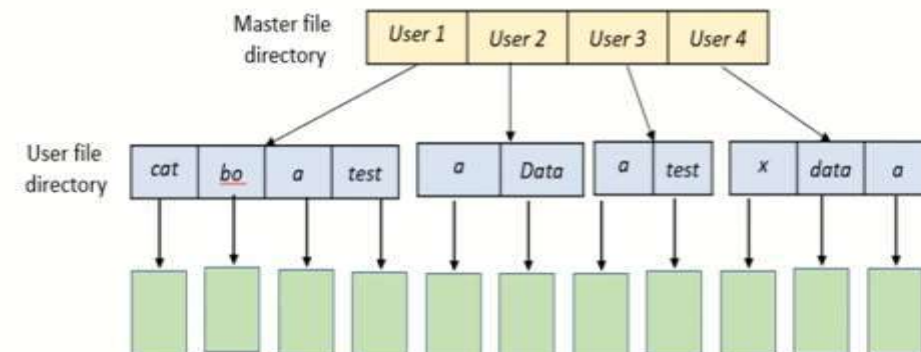


- Disadvantages: when number of files increases or when system has multiple users
 - Naming problem
 - Grouping problem

Two-Level Directory

Each user has own **user file directory (UFD)**

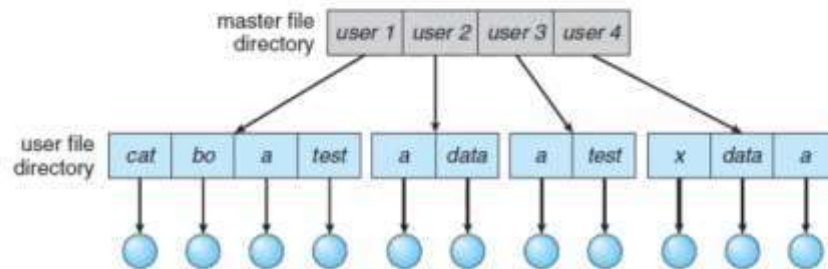
- UFDs have similar structures - each lists only files of a single user
- When user logs in - system's **master file directory (MFD)** searched
- MFD indexed by user name or account number - each entry points to UFD for that user



File Systems: File Directories

Two-level directory structure - Advantages

- Different users may have files with the same name
 - To create a file for a user, the operating system searches only that user's UFD, whether the name of the file is already existing or not.
 - To delete a file, the OS search and delete the file only in the local UFD;
 - thus, it **cannot accidentally delete another user's file** that has the same name.



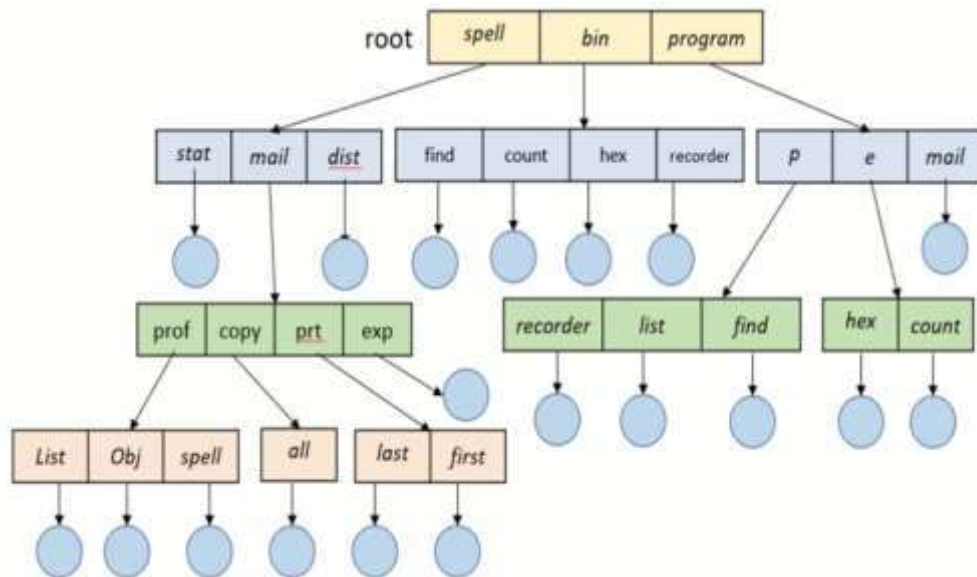
Two-Level Directory

- Disadvantages:
 - Isolates one user from another
 - Advantage when users are completely independent
 - Disadvantage when users want to cooperate on some task and to access one another's files
 - Grouping at user level not possible
- Two-level directory can be considered a tree of height 2
 - Root of tree is MFD - Direct descendants are UFDs
 - Descendants of UFDs are files - leaves of tree
 - Specifying user name and file name defines a **path** in tree from root (MFD) to a leaf (specified file)
 - Every file in system has path name

File Systems: File Directories

Tree-structured Directory

- Directory structure organized as a tree of arbitrary height
 - Allows users to create own subdirectories and to organize files accordingly
 - Has a root directory - every file in system has a unique path name



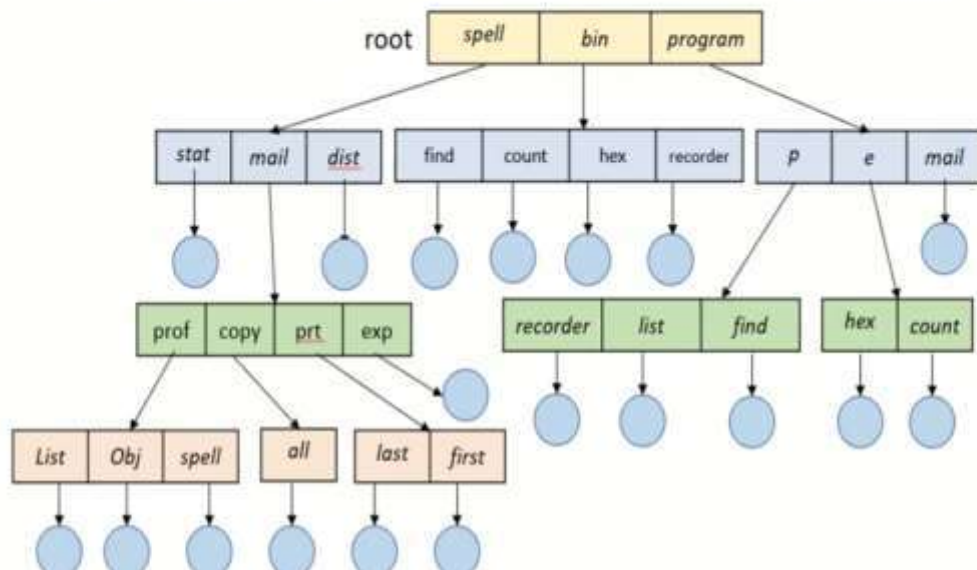
Tree-Structured Directories...

- A directory (or subdirectory) contains a set of files or subdirectories.
- All directories have the same internal format.
- Single bit is allocated to directory entries
 - Subdirectory=1; file=0
- Special **system calls** are used to create and delete directories.
- The **current directory** - should contain most of the files for the current process.
- An **absolute path name** begins at the root and follows a path down to the specified file.
- A **relative path name** defines a path from the current directory.

File Systems: File Directories

Tree-structured Directory

- Directory structure organized as a tree of arbitrary height
 - Allows users to create own subdirectories and to organize files according
 - Has a root directory - every file in system has a unique path name



Tree-structured Directory

- Path names can be of two types:
 - Absolute or relative
- In UNIX and Linux - absolute path name begins at root (designated by an initial "/")
 - Follows path down to specified file, giving directory names on path
- Relative path name defines path from current directory
- Ex. if current directory is /spell/mail
 - Relative path name prt/first refers to same file as absolute path name /spell/mail/prt/first

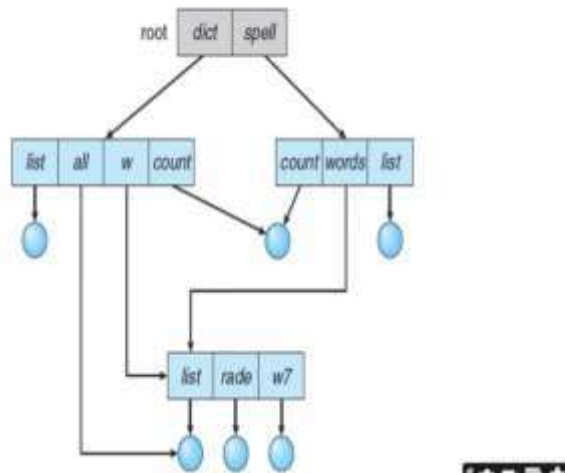
Drawback of Tree structured Directories

- The files cannot be shared between users, hence need to maintain multiple copies of same files in the system.
- Difficult to maintain
- Waste of memory

File Systems: File Directories

✓Acyclic-Graph Directories

- A tree structure prohibits the sharing of files or directories.
- An **acyclic graph**—that is, a graph with no cycles—allows directories to share subdirectories and files.
- The **same file or subdirectory** may be in two different directories.
- The acyclic graph is a natural generalization of the tree-structured directory scheme.



Acyclic-Graph Directories

- Implementation of shared files and subdirectories:
- **Approach 1** - Create a new directory entry called **link**
 - A pointer to another file or subdirectory - may be implemented as an absolute/relative path name
 - When reference to a file is made - search directory
 - If directory entry marked as a link - name of real file included in link information
 - Resolve link by using that path name to locate real file
 - Link clearly different from original directory entry
- **Approach 2** - duplicate all information both sharing directories
 - Both entries are identical and equal
 - Duplicate directory entries, make original and the copy indistinguishable

File Systems: File Sharing

File Sharing in an Operating System(OS) denotes how information and files are shared between different users, computers, or devices on a network; and files are units of data that are stored in a computer in the form of documents/images/videos or any others types of information needed.

For Example:

- Suppose letting your computer talk to another computer and exchange pictures, documents, or any useful data.
- This is generally useful when one wants to work on a project with others, send files to friends, or simply shift stuff to another device.
- Our OS provides ways to do this like email attachments, cloud services, etc. to make the sharing process easier and more secure.
- Now, file sharing is nothing like a magical bridge between Computer A to Computer B allowing them to swap some files with each other.

File Systems: File Sharing

Primary Terminology Related to File Sharing

Let's discuss those primary terminologies first:

- **Folder/Directory:** It is basically like a container for all of our files on a computer. The folder can contain files and even other folders maintaining like hierarchical structure for organizing data.
- **Networking:** It is involved in connecting computers or devices where we need to share the resources. Networks can be local (LAN) or global (Internet).
- **IP Address:** It is numerical data given to every connected device on the network
- **Protocol:** It is given as the set of rules which drives the communication between devices on a network. In the context of file sharing, protocols define how files are transferred between computers.
- **File Transfer Protocol (FTP):** FTP is a standard network protocol used to transfer files between a client and a server on a computer network.

File Systems: File Sharing

- In a multiuser system, there is almost always a requirement for allowing files to be shared among a number of users. Two issues arise: access rights and the management of simultaneous access.

a. Access Rights-

- The file system should provide a flexible tool for allowing extensive file sharing among users.
- The file system should provide a number of options so that the way in which a particular file is accessed can be controlled.
- Typically, users or groups of users are granted certain access rights to a file. A wide range of access rights has been used.

File Systems: File Sharing

a. Access Rights-

- The following list is representative of access rights that can be assigned to a particular user for a particular file:
 - None – User cannot see or access the file.
 - Knowledge – User knows the file exists and its owner.
 - Execution – User can run the program but cannot copy it.
 - Reading – User can read and copy the file.
 - Appending – User can add data but cannot modify existing data.
 - Updating – User can modify, add, or delete data.
 - Changing Protection – User can change access rights.
 - Deletion – User can remove the file.

File Systems: File Sharing

a. Access Rights-

- One user is designated as owner of a given file, usually the person who initially created a file.

The owner has all of the access rights listed previously and may grant rights to others. Access can be provided to different classes of users:

1. Specific user: Individual users who are designated by user ID
2. User groups: A set of users who are not individually defined. The system must have some way of keeping track of the membership of user groups.
3. All: All users who have access to this system. These are public files.

File Systems: File Sharing

b. Simultaneous Access -

- In a multiuser system, multiple users may access the same file simultaneously.
- When users have append or update permission, the system must control access to avoid conflicts.
- File Locking (Brute-force approach):
 - Entire file is locked when one user updates it.
 - Other users must wait until the lock is released.
- Record Locking (Fine-grained control):
 - Only specific records of the file are locked during update.
 - Allows other users to access different records simultaneously.
- The system must handle mutual exclusion and avoid deadlocks while managing shared access.

File Systems: File Sharing achieve through

Sharing of files on multi-user systems is desirable

i) Multiple users

How to implement sharing and **protection** –default and specific access

- ▶ Systems use the concept of **file owner ,group , all other users**.
- ▶ If multi-user system
 - **Owner IDs** identify users, allowing permissions and protections to be per-user.(user id compared with the owner attribute)
 - **Group IDs** allow users to be in groups, permitting group access rights
- ▶ Remote file systems uses networking to share file systems spread across systems around the world.
- ▶ Remote file sharing methods are:
 - Manually transfer files between machines via programs like **FTP**
 - **Distributed file systems**-one or more central servers store files that can be accessed, with proper authorization rights, by any number of remote clients in the network.
 - Semi automatically via the **world wide web**

File sharing:

In Multi-user system, there is always a requirement for sharing the files, among many number of users.

Two **Issues** may arise while Sharing, **They are:**

1. providing access rights.
2. Management of simultaneous access.

File Systems: File Sharing achieve through

2.Remote file system :

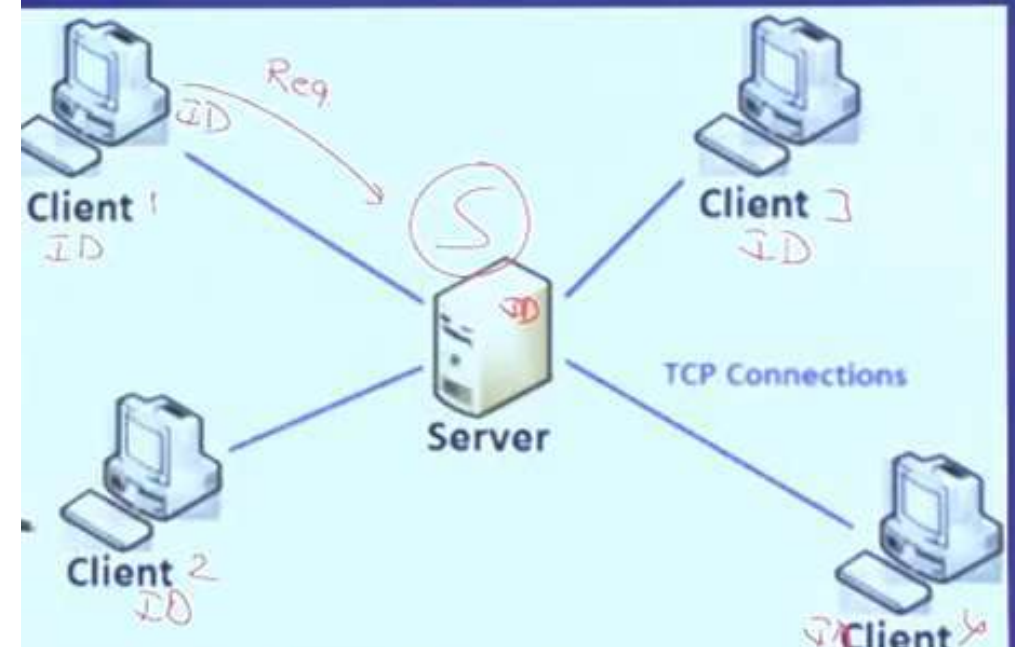
Communication is done between the systems which are located far away.

Communication is nothing but sending & receiving a file.

File communication can be done by two methods:

1. Client-Server Model
2. Distribution Information System Model

Client-Server Architecture



File Systems: File Sharing achieve through

2. Distribution Information System Model (DIS)

The Disadvantages of Client-Server model can be Overcome by this Distribution Information System Model.

This Model also known as Distributes Naming Services.

There are TWO Systems in this model. They are:

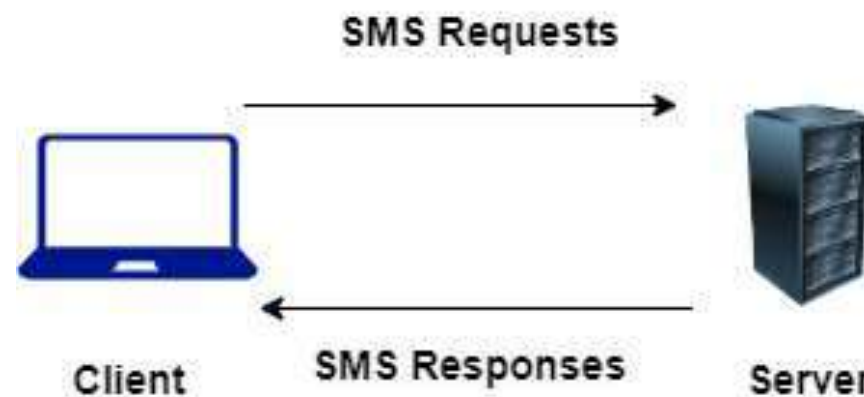
1. Network Information Service (NIS)
2. Common Internet File System (CIFS)

File Systems: File Sharing achieve through

1. Server Message Block (SMB) / Common Internet File System (CIFS)

SMB is like a network based file sharing protocol mainly used in windows operating systems. It allows our computer to share files/printer on a network. SMB is now the standard way for seamless file transfer method and printer sharing.

Example: Imagine in a company where the employees have to share the files on a particular project . Here SMB is employed to share files among all the windows based operating system.orate on projects. SMB/CIFS is employed to share files between Windows-based computers. Users can access shared folders on a server, create, modify, and delete files.



File Systems: File Sharing

In the 1990s, Microsoft updated the Server Message Block (SMB) protocol and named it as Common Internet File System (CIFS). The CIFS application uses the TCP/IP protocol and works best on Windows operating systems to share files between machines on a network.

Initially, incorporated for file sharing, but now CIFS is exercised for network management and other network services.

Cons of CIFS

- It supports the N number of commands and subcommands for file sharing but faces maintenance issues with these commands
- It has a less data storage size on comparing with SMB
- It has a complicated configuration process
- The client or user may find it difficult to diagnose and find faults in CIFS
- It is less scalable than NFS
- The communication speed in Common Internet File System (CIFS) is moderate

Pros of CIFS

- It allows a secured file transfer within the network
- No requirement for an additional external transport protocol for transferring SMB messages from clients to the server
- It is easy to edit, read, write, and remove files or print queues
- It supports record tracking
- It allows easy access to the files over the network
- It updates the client or user via notifications when file contents are modified over a network
- It supports a safe file cache, non file system attributes, LDAP services, record locking

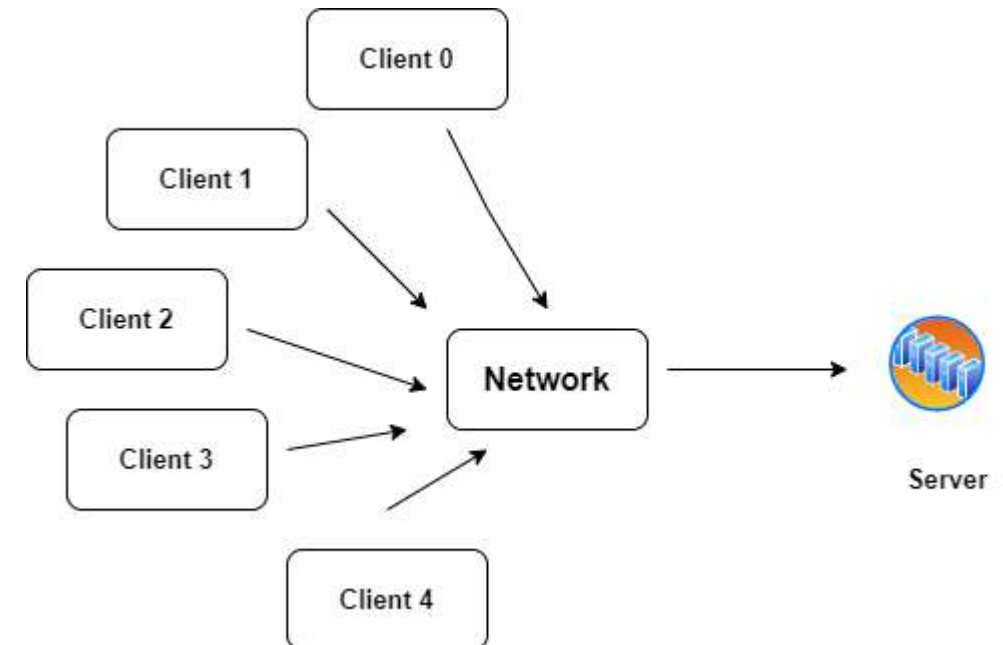
File Systems: File Sharing achieve through

2. Network File System (NFS) NFS is a distributed based file sharing protocol mainly used in Linux/Unix based operating System. It allows a computer to share files over a network as if they were based on local. It provides a efficient way of transfer of files between servers and clients.

Example: Many Programmer/Universities/Research Institution uses Unix/Linux based Operating System. The Institutes puts up a global server datasets using NFS. The Researchers and students can access these shared directories and everyone can collaborate on it.

Sun Microsystems introduced Network File System (NFS) in the 1980s for file sharing and remote communication amid servers and end user machines and devices. The easy to use file system allows users to edit, view, read and write files remotely on a system.

Iso, the user or administrator has access to organize and mount the file system through NFS. NFS is affordable and provides better locking and caching performance.



File Systems: File Sharing achieve through

2. Network File System (NFS)

Pros of NFS

- It is an affordable solution for network file sharing
- It uses existing Internet Protocol infrastructure and is easy to install
- It has a Centralized File Sharing System that reduces the system admin overhead
- You require no additional disk space or software in NFS
- It provides easy access to remote files
- It supports NFS mapped files and the cache file system
- It supports a file locking system
- It has a transparent mounting process
- Less requirement for removable media storage for security purposes
- It is a consistent and reliable file sharing protocol

Cons of NFS

- Network File System supports the inherently insecure Remote Procedure Calls (RPC) communication which must be allowed only on a trusted network behind a firewall
- NFS versions, NFSv4 and NFSv4.1, have limited maximal bandwidth. As a result, on noticing heavy traffic loads, NFS may go slow
- The file locking and caching system for configuration and installation of shared file access can be a daunting task
- NFS does not provide Requirement sessions
- It includes no special security features in NFS
- It has a limited block size needed when transferring a read or write request
- In some cases, users find it difficult to access files at the same time

File Systems: File Sharing achieve through

NFS and CIFS Ports and Protocols

CIFS Ports

CIFS uses the following ports:

- For TCP – 139 and 445 port
- For UDP – 138 and 137 port



NFS uses the same port for both TCP and UDP in each case:

- For Server – 111 port
- For operations – 2049 port
- For checking client status – 1110 port
- To access NFS lock manager 4045 port

Security

CIFS supports advanced security features and is highly reliable on comparing with NFS.

There are no special security features for NFS and is not a reliable protocol on comparing with CIFS.

Speed

CIFS – Moderate communication speed
NFS – High communication speed.

LDAP services

CIFS allows the use of LDAP services for storing and accessing the database.

NFS allows the use of LDAP services to view the information.

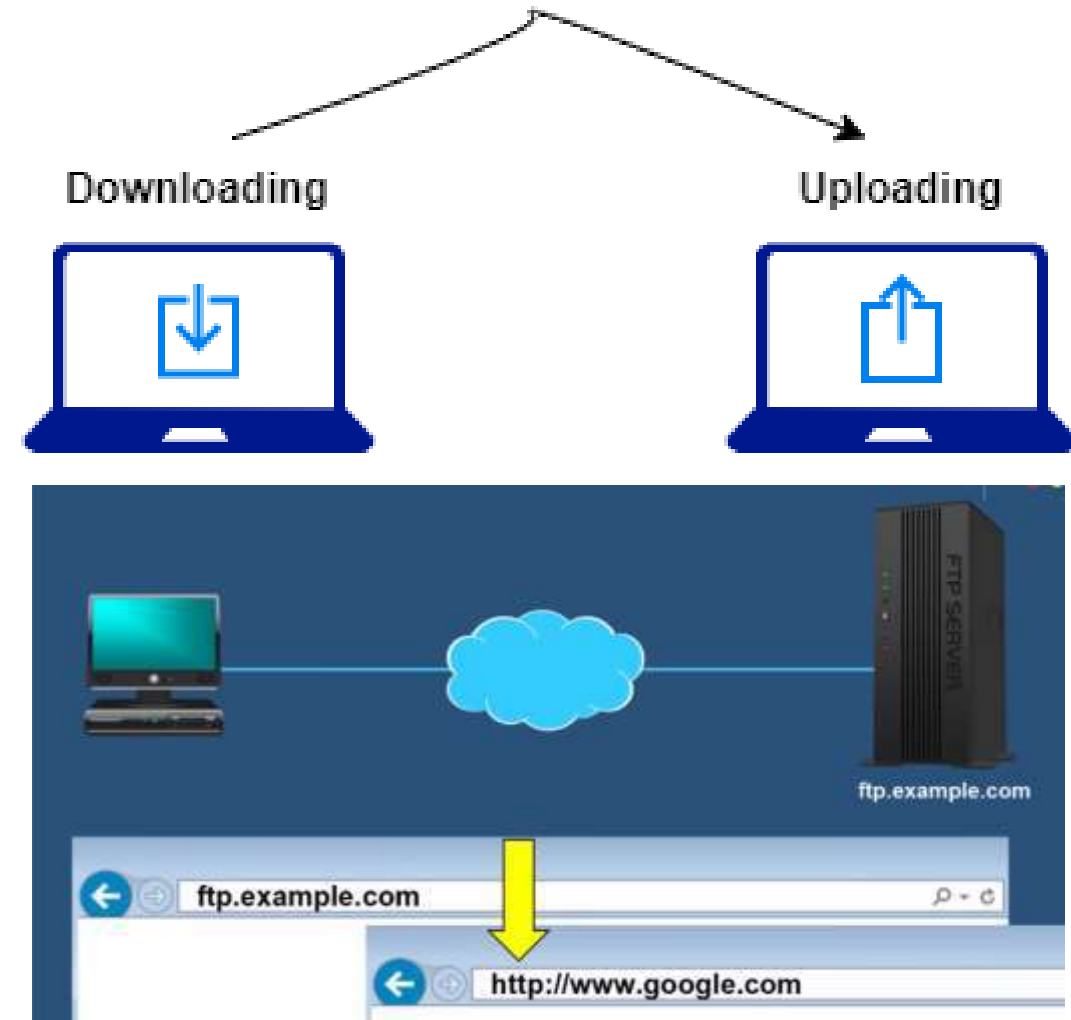
File Systems: File Sharing achieve through

3. File Transfer Protocol (FTP)

It is the most common standard protocol for transferring of the files between a client and a server on a computer network. FTPs supports both uploading and downloading of the files, here we can download,upload and transfer of files from Computer A to Computer B over the internet or between computer systems.

Example: Suppose the developer makes changes on the server. Using the FTP protocol, the developer connects to the server they can update the server with new website content and updates the existing file over there.

File Transfer Protocol



File Systems: File Sharing achieve through

3. File Transfer Protocol (FTP)

Every FTP server has an address, which looks similar to an IP address or URL. It can take the form of words, such as **ftp.sharemyfiles.net**, or a series of numbers, such as 123.456.789.

Using FTP to share files involves two things:

- 1. Uploading them to the server so someone else can access them**
- 2. Downloading them from the server**

For example, you might upload files to an FTP server to use on a webpage. Or your colleague might upload a video file to the FTP server for you. You can then log into the FTP client to download it to your device.

Email

Sending files over email is another example of a client-server file sharing system. To send a file over email, you attach it to a message. You can send the same file to multiple recipients simultaneously.

There are several major drawbacks to using email to share files, including limitations on file size, version-control issues, redundancy of files, difficulty of oversight by IT, and other security concerns.

File Systems: File Sharing achieve through

4. Cloud-Based File Sharing

It involves the famous ways of using online services like Google Drive, DropBox , One Drive ,etc. Any user can store files over these cloud services and they can share that with others, and providing access from many users. It includes collaboration in realtime file sharing and version control access.

Ex: Several students working on a project and they can use Google Drive to store and share for that purpose. They can access the files from any computer or mobile devices and they can make changes in realtime and track the changes over there.



File Systems: File Sharing achieve through

4. Cloud-Based File Sharing

- Compared to other file sharing programs, cloud-based solutions allow for faster uploads and downloads. When you upload something to a P2P network or FTP server, you're limited by your internet connection speed.
- The same is true for downloading from a P2P program or FTP server. Since you aren't actually downloading the file when you use a cloud-based program, you don't have to wait for it to transfer from the server to your device.



Type of file sharing software	How it works
Local system file sharing	Built-in functionality within operating systems to share files and folders on a local network
Client-server file sharing	Dedicated server stores files, while clients (computers) connect to the server to upload, download, and manage files
P2P file sharing	Devices directly connect to each other to share files without a central server
Cloud file sharing	Files are stored on remote servers accessed over the internet

File Systems: Record Blocking in OS

- Records are the logical unit of a file whereas blocks are units of I/O with secondary storage.
- It performs for I/O and the records must organize as blocks. Blocking is a process of grouping similar records into blocks that the operating system component explores exhaustively.
- Block is the unit of I/O for secondary storage. Generally, the larger blocks reduce the I/O transfer time.
- Larger blocks require larger I/O buffers. In record blocking, the records are grouped into blocks by shared properties that are indicators of duplication.

Types of Record Blocking in OS

Generally, there are three types of record-blocking methods.

1. Fixed blocking
2. Variable-length spanned blocking
3. Variable-length unspanned blocking

File Systems: Record Blocking in OS

Generally, there are three types of record-blocking methods.

1. Fixed blocking-In this method, **record lengths are fixed**. The prescribed number of records stored in a block. **Internal fragmentation is stored in a block**. Fixed blocking is common for sequential files with fixed-length records length records

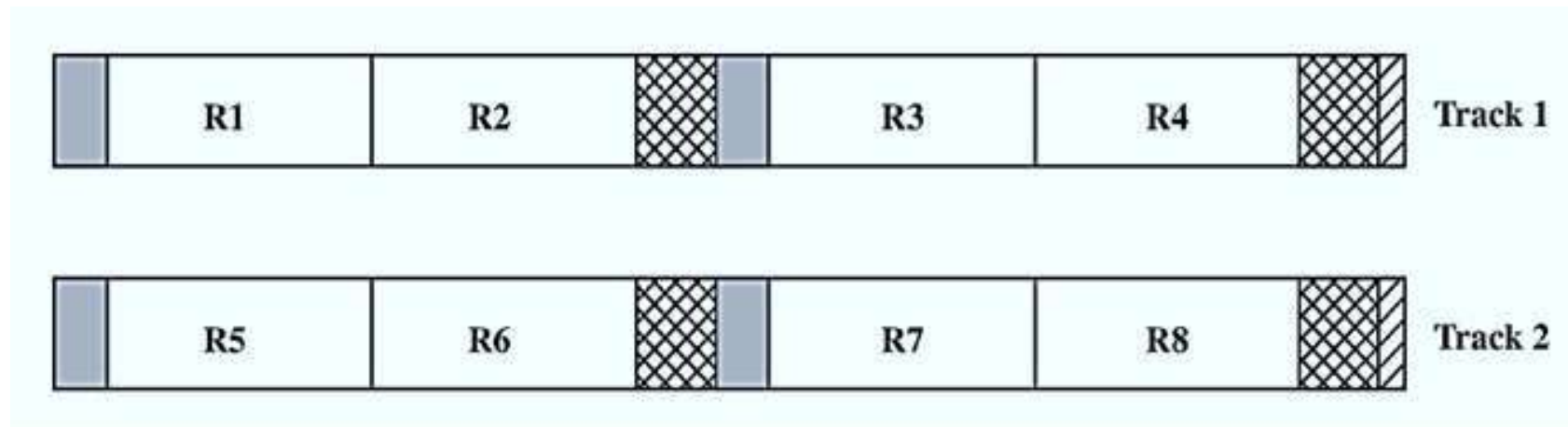


Fig: Fixed Blocking

File Systems: Record Blocking in OS

Generally, there are three types of record-blocking methods.

2. Variable-length spanned blocking-In this method, **record sizes aren't same, Variable-length records packing into blocks with no unused space**. So, some records may divide into two blocks. In this type of situation, a pointer passes from one block to another block.

So, the Pointers used to span block unused space. It is efficient in length and efficiency of storage. It doesn't limit record size. It is more difficult to update

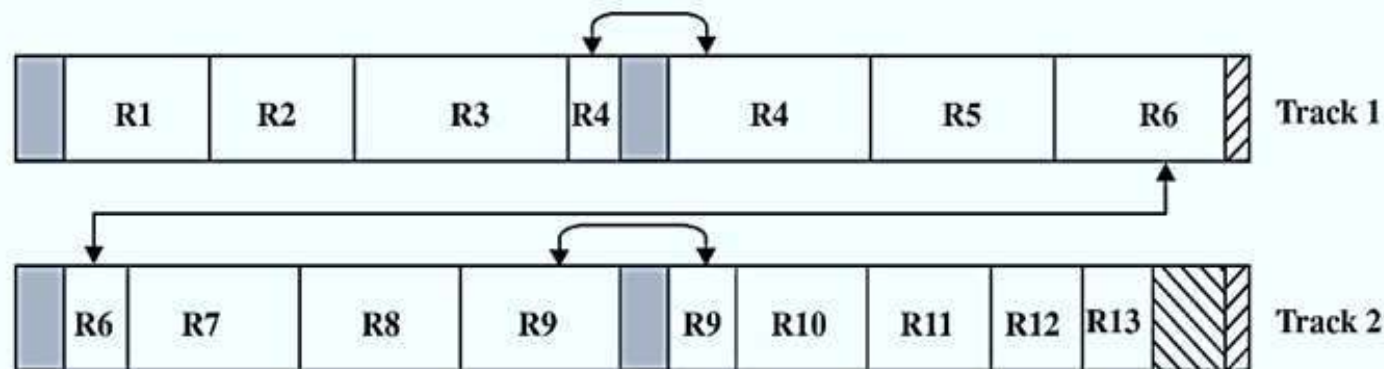


Fig: Variable-length spanned blocking

File Systems: Record Blocking in OS

Generally, there are three types of record-blocking methods.

3. Variable-length unspanned blocking-Here, records are variable in length, but the records span between blocks. In this method, the wasted area is a serious problem, because of the inability to use the remainder of a block, if the next record is larger than the remaining unused space.

These blocking methods result in blocking, results in wasted space and limit record size to the size of the block

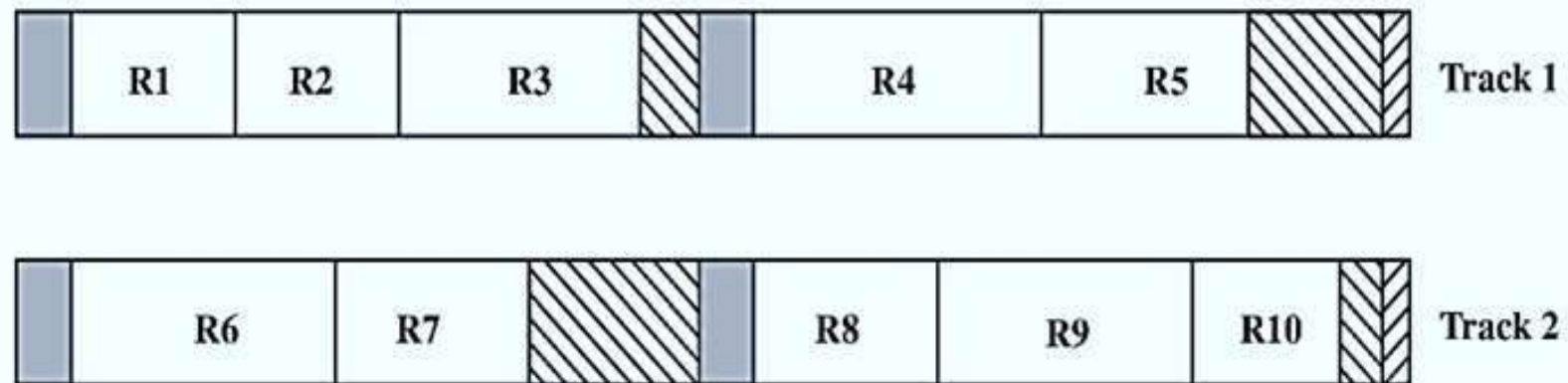
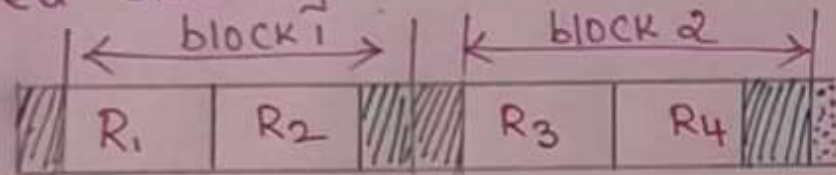


Fig: Variable-length unspanned blocking

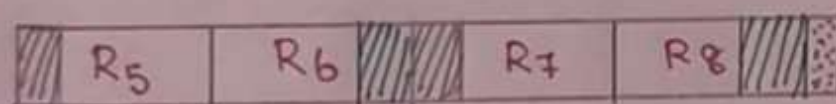
File Systems: Record Blocking in OS

- ① fixed blocking: internal fragmentation
- ② variable length spanned blocking: some records must span in two blocks
- ③ variable length unspanned blocking: waste space in part of the block

Fixed blocking

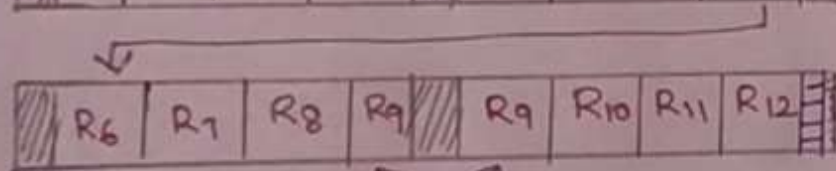
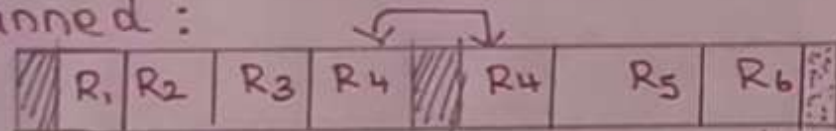


Track 1

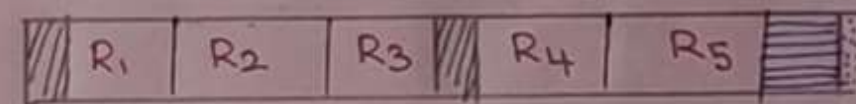


Track 2

Spanned :



unspanned blocking



■ Gaps

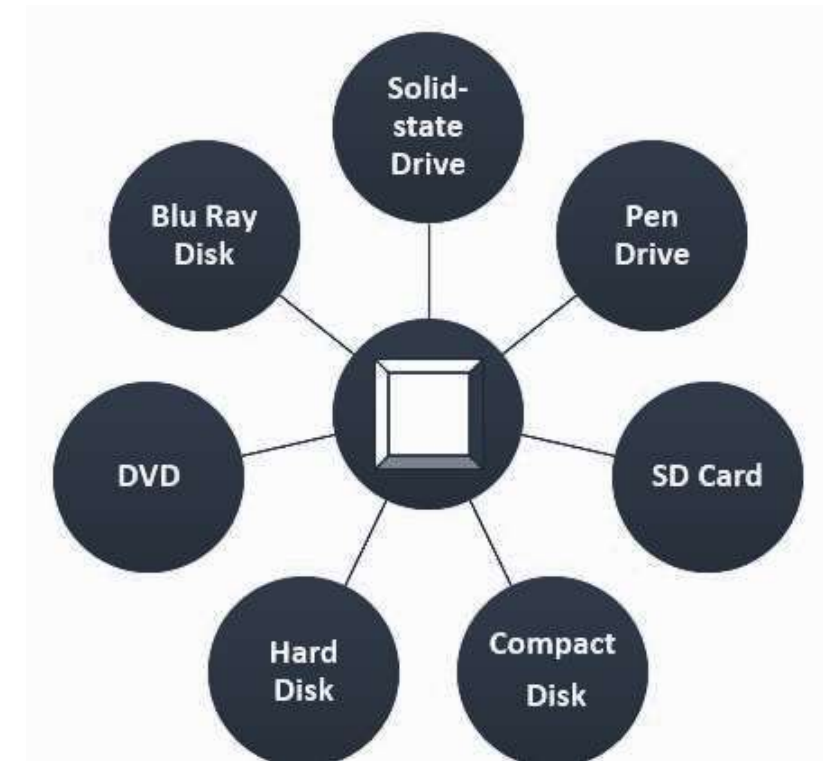
■ waste due to fixed block size

■ waste due to block fit to track

■ waste due to record fit to block

File Systems: SECONDARY STORAGE MANAGEMENT

- A file stored on secondary storage (disk) is made up of multiple blocks.
- Secondary storage devices are non-volatile, long-term data storage solutions, including Magnetic (HDDs, Tapes), Solid-State (SSDs, USB drives), and Optical (CDs, DVDs) devices.
- The Operating System (OS) or File Management System allocates these blocks to files.
- Two important management issues arise.
 - **Space Allocation:** The OS must decide how disk space (blocks) is assigned to files.
 - **Free Space Management:** The system must track which blocks are free and available for new files.
- Relationship Between Allocation and Free Space-The method used for file allocation affects how free space is managed.



File Systems: SECONDARY STORAGE MANAGEMENT

File Allocation refers to how the Operating System assigns disk blocks to a file in secondary storage.

Main Issues in File Allocation:

1. Space Allocation Time

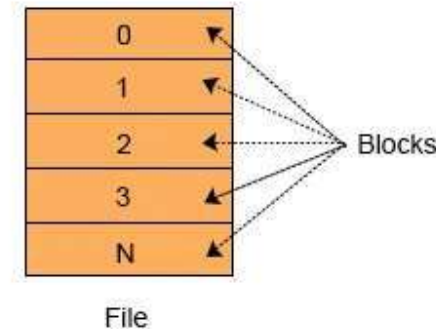
- Decide whether the maximum space for a file is allocated when the file is created or allocated gradually as the file grows.

2. Portion Size

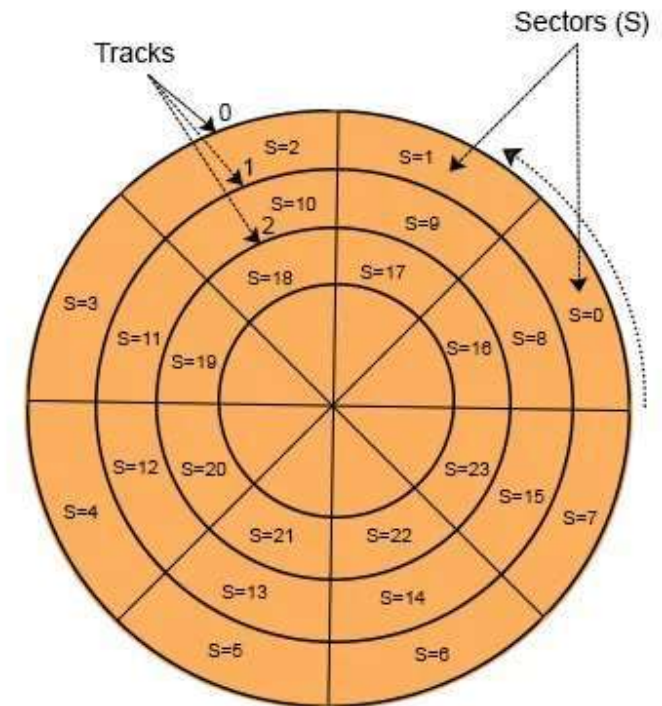
- Disk space is allocated in contiguous units called portions.
- A portion is a continuous group of blocks.
- The portion size may vary from a single block to the entire file.

3. Allocation Tracking

- The system must maintain a data structure or table to track which blocks belong to which file.
- Example: File Allocation Table (FAT) used in systems like MS-DOS.



File-block Size = Disk-Sector size



Hard Disk with 3-tracks
each track contains 8-sectors

File Systems: SECONDARY STORAGE MANAGEMENT

Let us examine these issues in turn.

1. Preallocation vs Dynamic Allocation

Preallocation

- In preallocation, the maximum file size is specified when the file is created.
- The system allocates all required disk space in advance.
- Suitable when the file size can be estimated (e.g., program compilation, data transfer).
- Disadvantage-May cause wastage of storage space if the estimated size is larger than the actual file size.

Dynamic Allocation

- Disk space is allocated gradually as the file grows.
- Space is given in portions when required.
- Advantage- Better utilization of disk space and reduces wastage.

File Systems: SECONDARY STORAGE MANAGEMENT

2. Portion Size in File Allocation

In file allocation, a portion is a contiguous group of disk blocks allocated to a file. The size of the portion can range from a single block to the entire file. Choosing the appropriate portion size involves a trade-off between the performance of a single file and the overall efficiency of the system.

If large contiguous portions are used, the file is stored in one continuous space. This improves performance and sequential access speed. It also reduces the size of allocation tables. However, it becomes difficult to reuse free space.

If small fixed-size blocks are used, disk space is allocated block by block. This method provides greater flexibility and easier space allocation, but it may require larger allocation tables to manage all blocks.

Some factors must be considered when choosing portion size:

- Contiguous storage improves performance for sequential operations.
- Many small portions increase the size of allocation tables.
- Fixed-size blocks make re-allocation easier.
- Variable or small portions reduce wastage of storage space.

File Systems: SECONDARY STORAGE MANAGEMENT

2. Portion Size in File Allocation -Of course, these items interact and must be considered together. The result is that there are two major alternatives:

1. Variable Large Contiguous Portions

- Large continuous disk space allocated to a file.
- Advantages
 - Better performance.
 - Smaller allocation tables.
 - Less storage waste.
- Disadvantage-Difficult to reuse free space.

2. Blocks (Small Fixed Portions)

- File is stored in small fixed-size blocks.
- Advantages-Greater flexibility in allocation.
- Disadvantages
 - Requires large tables or complex allocation structures.
 - Contiguous storage is not guaranteed.

File Systems: SECONDARY STORAGE MANAGEMENT

Both large contiguous portions and block allocation methods can be used with preallocation or dynamic allocation.

In the case of variable large contiguous portions, the file is preallocated as one continuous group of disk blocks. In this method, there is no need for a complex file allocation table. The system only needs:

- a pointer to the first block, and
- the total number of blocks allocated.

In the case of block allocation, disk space is divided into small fixed-size blocks. All required blocks for the file are allocated at one time, and a File Allocation Table (FAT) is used to keep track of them. Since the number of blocks is fixed, the allocation table size also remains fixed.

File Systems: SECONDARY STORAGE MANAGEMENT

When variable-size portions are used, a problem called **fragmentation** may occur. Fragmentation means free disk space is divided into small scattered pieces, making it difficult to allocate large contiguous space for new files. This problem is similar to fragmentation in partitioned main memory.

To manage free space, the system may use strategies such as:

- **First Fit:** Select the first available free space that is large enough.
- **Best Fit:** Select the smallest free space that satisfies the requirement.
- **Nearest / Next Fit:** Select free space closest to the previous allocation to improve locality.

It is not clear which strategy is best. The difficulty in modeling alternative strategies is that so many factors interact. Performance depends on several factors:

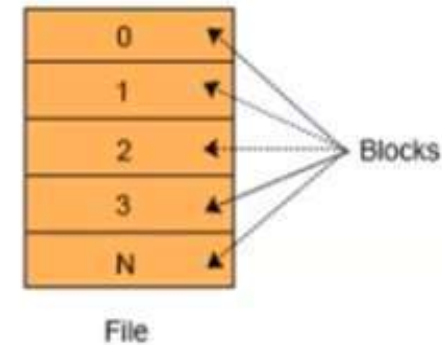
- Type of files
- File access patterns
- Degree of multiprogramming
- System performance
- Disk caching
- Disk scheduling



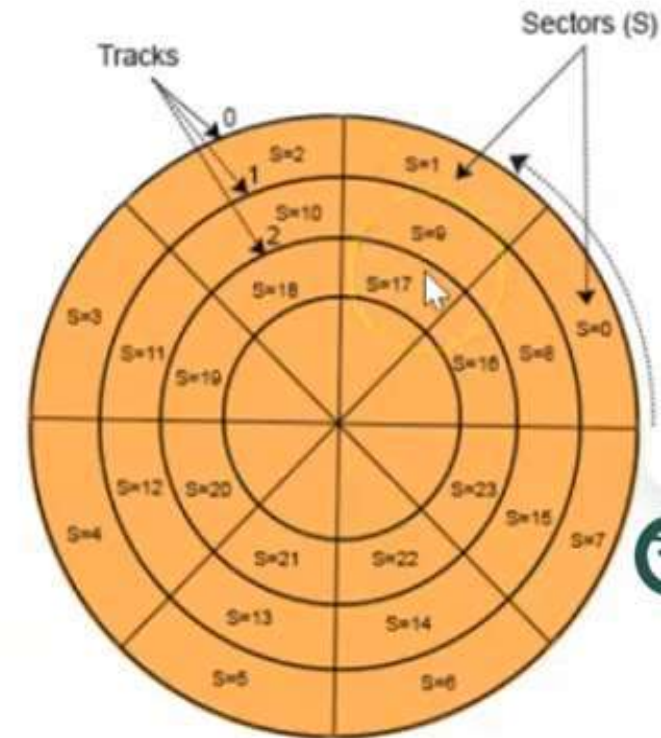
File Systems: SECONDARY STORAGE MANAGEMENT

❑ File Allocation

- File Allocation defines **how the files are stored in the disk blocks.**
- File allocation methods are used to place the files on the hard disk and access them from the disk as required.
- **Hard disk** is divided into several physical **sectors.**
- **File** is divided into several logical **blocks.**
- $\text{Size}(\text{file's block}) = \text{Size}(\text{Hard disk's sector}).$
- So that blocks of files could be loaded into the hard disk sectors.



File-block Size = Disk-Sector size



Hard Disk with 3-tracks
each track contains 8-sectors

File Systems: SECONDARY STORAGE MANAGEMENT

Contiguous Allocation

Contiguous allocation stores each file as a contiguous sequence of blocks on disk. The file directory entry contains the address of the starting block and the length of the file.

- ✓ Simple implementation with direct access
- ✓ Excellent read performance (sequential access)
- ✗ External fragmentation
- ✗ File growth is difficult

1	2	3	4	5	6	7	8
A1	A2	A3	A4	A5	14	15	16
17	18	19	20	21	22	23	24
B1	B2	B3	28	29	30	31	32
33	34	35	36	37	38	39	40
C1	C2	C3	C4	C5	C6	47	48
49	50	51	52	53	54	55	56
57	58	59	60	61	62	63	64

■ File A ■ File B ■ File C

File Systems: SECONDARY STORAGE MANAGEMENT

Contiguous Allocation - Directory Structure

File Name	Starting Block	Length (blocks)	Access Rights
File A	9	5	<i>rw-r--r--</i>
File B	25	3	<i>rw-rw-r--</i>
File C	41	6	<i>rw-r--r--</i>

In contiguous allocation, the directory entry for each file contains just the **starting block address** and **length**. This simplicity allows for efficient direct access to any block within the file.

```
// Accessing the kth block of File A
int position = startingBlock + (k - 1);

// For File A, to access block 3:
position = 9 + (3 - 1); // position = 11
```

example on contiguous File Allocation

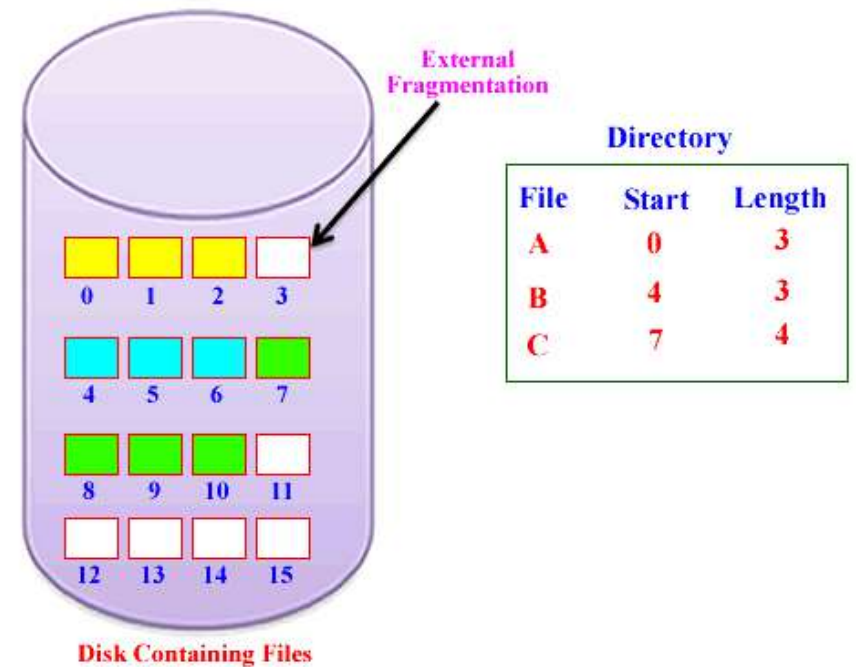


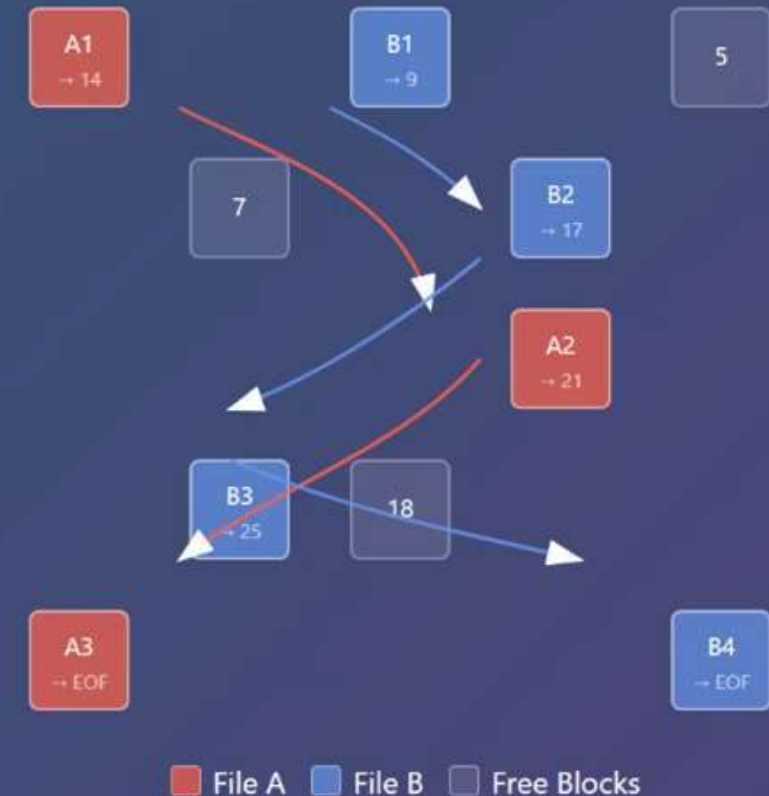
Fig: Showing Contiguous Allocation of Disk

File Systems: SECONDARY STORAGE MANAGEMENT

Linked Allocation

Linked allocation stores each file as a linked list of blocks. Each block contains a pointer to the next block in the file. The directory entry contains pointers to the first and last blocks.

- ✓ No external fragmentation
- ✓ File size can grow dynamically
- ✗ Random access is slow (sequential traversal)
- ✗ Space overhead for pointers
- ✗ Reliability issues (broken links)



File Systems: SECONDARY STORAGE MANAGEMENT

Linked Allocation - File Allocation Table (FAT)

The File Allocation Table (FAT) is a popular implementation of linked allocation used in many file systems including MS-DOS and early Windows.

Instead of storing pointers with each disk block, FAT keeps all the links in a separate table in memory. Each entry in the FAT corresponds to one disk block and contains the number of the next block in the file.

Key advantages:

- Random access is improved (no disk reads for pointers)
- Entire FAT can be cached in memory
- No space wasted in data blocks for pointers

Index	0	1	2	3	4	5	6	7	8	9
Value	0	0	0	4	5	-1	9	0	0	10

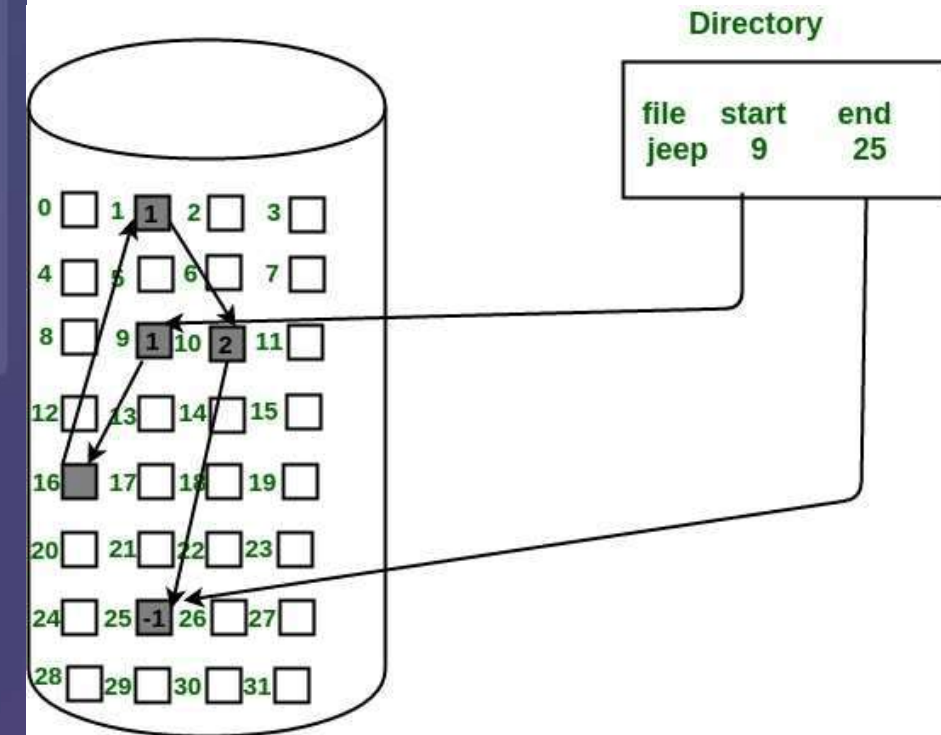
Index	10	11	12	13	14	15	16	17	18	19
Value	12	0	-1	0	0	0	0	0	0	0

■ File A ■ File B ■ End of File (-1) ■ Free Block (0)

To access File A: Start at block 3, follow pointers (3→4→5→EOF)

File Name	Starting Block	Size
File A	3	3 blocks
File B	6	4 blocks

Example on Linked File Allocation



The file 'jeep' in following image shows how the blocks are randomly distributed.
The last block (25) contains -1 indicating a null pointer and does not point to any other block.

File Systems: File System Security

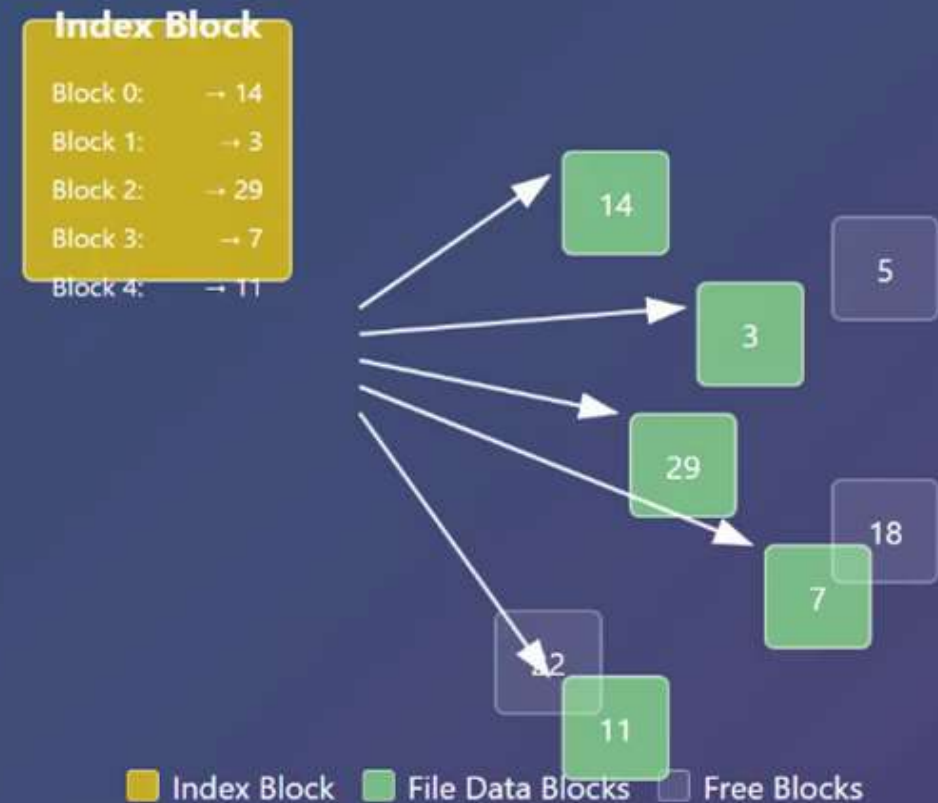
Indexed Allocation

Indexed allocation uses a special index block that contains pointers to all the blocks of a file. The directory entry points to this index block.

- ✓ Supports direct access to any block
- ✓ No external fragmentation
- ✓ File size can grow dynamically
- ✗ Space overhead for index blocks
- ✗ Maximum file size limited by index block size

To access the kth block of a file:

1. Access the index block
2. Find the kth entry in the index
3. Access the data block pointed to by that entry

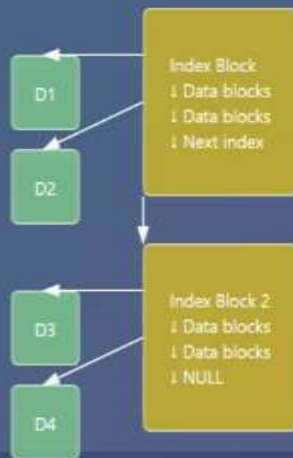


File Systems: SECONDARY STORAGE MANAGEMENT

Indexed Allocation Schemes

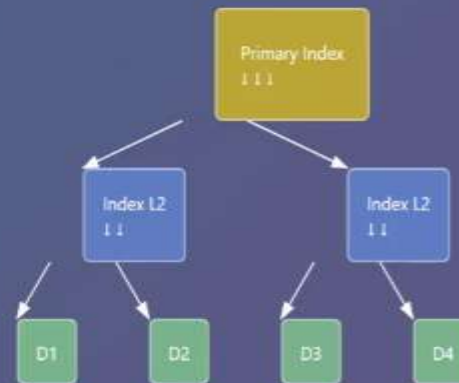
Linked Scheme

Index blocks are linked together to support larger files.

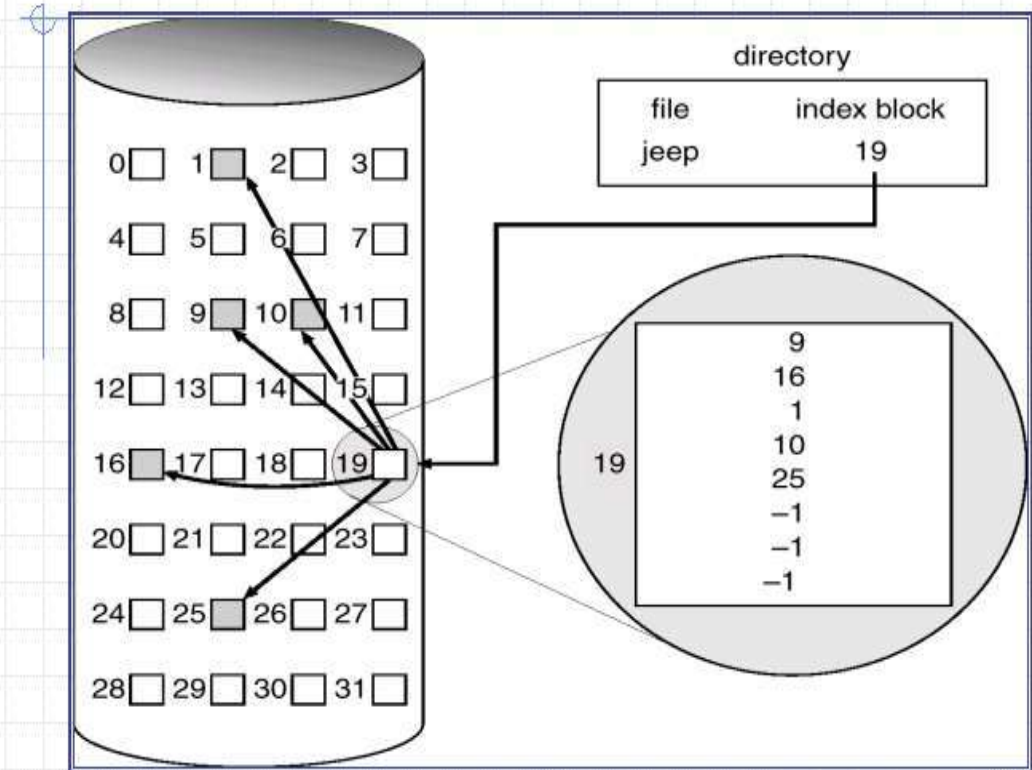


Multilevel Scheme

Uses a tree of index blocks with multiple levels of indirection.



Example of Indexed Allocation



os12

22

File Systems: SECONDARY STORAGE MANAGEMENT

Comparison of File Allocation Methods

Method	Random Access	Space Efficiency	File Growth	Fragmentation	Implementation
Contiguous	Excellent $O(1)$	No overhead	Difficult Requires relocation	External fragmentation	Simple
Linked	Poor $O(n)$	Pointer overhead	Easy Dynamic growth	No external fragmentation	Moderate
FAT (Linked)	Good $O(n)$ but cached	Table overhead	Easy Dynamic growth	No external fragmentation	Moderate
Indexed	Excellent $O(1)$	Index block overhead	Easy with multi-level	No external fragmentation	Complex

Real-world file systems often use combinations of these methods to balance their strengths and weaknesses. For example, ext4 uses a variant of indexed allocation, NTFS uses a master file table (similar to FAT but more complex), and FAT32 uses the File Allocation Table approach.

File Systems Security

After a user successfully logs into the system, the user is given access to certain hosts and applications. However, this is not enough for systems that store sensitive data. Therefore, user-access control is used to provide additional security.

In user-access control, each user is identified by the system, and a user profile is created. This profile specifies the allowed operations and file access permissions for that user. The Operating System (OS) enforces these rules based on the user profile.

For example, in an organization:

- All employees in the administration department may be allowed to view a list of company personnel.
- But only authorized individuals can access salary information.

In some systems, such as a Database Management System (DBMS), security checks are more detailed. Unlike the OS, which may check permission only once when accessing a file or application, the DBMS checks permission for every data access request.

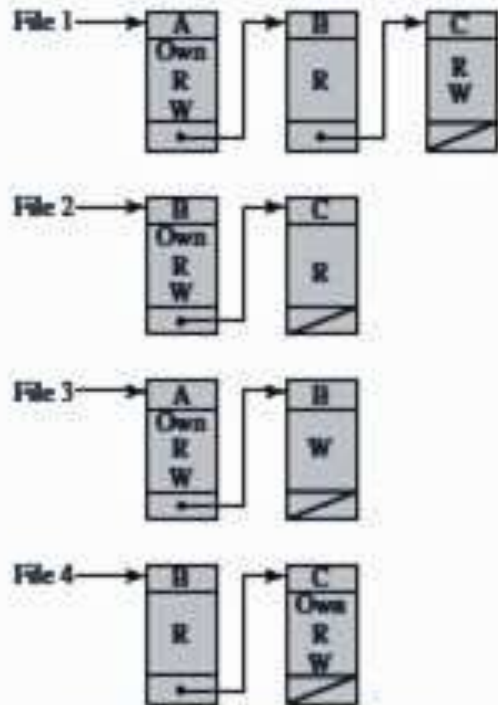
The access decision depends on:

- User identity
- Specific data being accessed
- Information already provided to the user

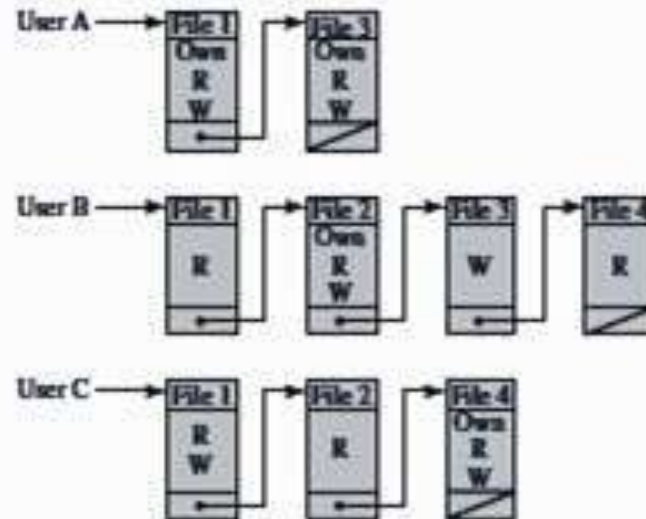
File Systems Security

	File 1	File 2	File 3	File 4	Account 1	Account 2
User A	Own R W		Own R W		Inquiry credit	
User B	R	Own R W	W	R	Inquiry debit	Inquiry credit
User C	R W	R		Own R W		Inquiry debit

(a) Access matrix



(b) Access control lists for files of part (a)



(c) Capability lists for files of part (a)

Figure 12.15 Example of Access Control Structures

A common model used for controlling access is the **Access Matrix**, which defines which users can access which resources and what operations they are allowed to perform. shown in Figure 12.15a)

Filesystem Access Control Lists (ACLs) are security mechanisms that define specific permissions (read, write, execute) for users or groups on files and directories, acting as detailed access rules.

They enable granular control beyond standard owner/group/others permissions, allowing precise management of "inquiry debit" (read/write/modify) and "inquiry credit" (append/create) actions on sensitive data objects

File Systems Security

Access Control Model in File System Security: Basic Elements of Access Control Model

1. Subject

- A subject is an entity that can access system resources.
- Usually represented by a process acting on behalf of a user or application.

2. Object

- An object is anything whose access needs to be controlled.
- Examples: files, parts of files, programs, memory segments, database records, software objects. (e.g., Java objects).

3. Access Rights

- Defines how a subject can access an object.
- Common rights include read, write, execute, update, and delete.

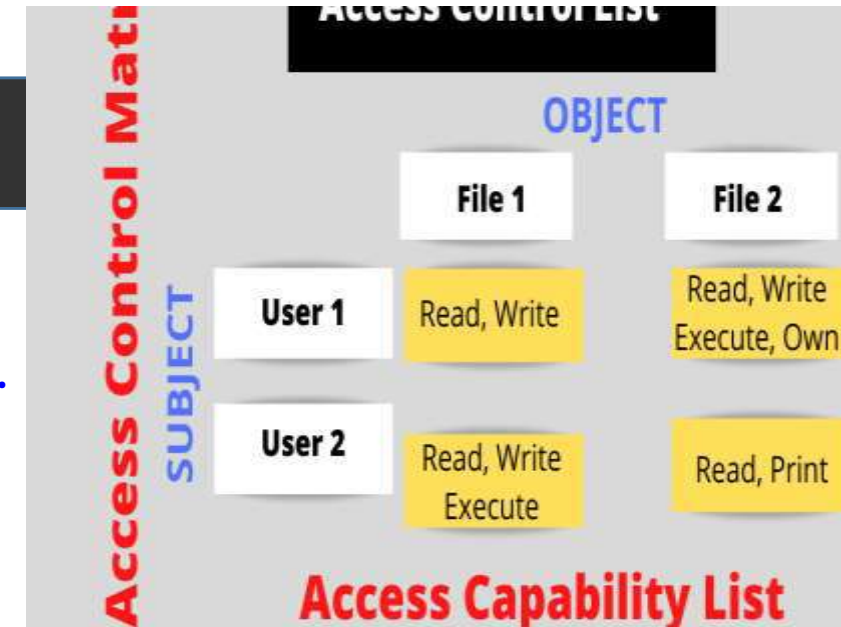
Access Matrix

- Access control is often represented using an Access Matrix.
- Rows represent subjects (users or processes).
- Columns represent objects (files, programs, data).
- Each cell in the matrix shows the access rights of a subject for a particular object.

Implementation of Access Matrix

File Systems Security

1. **Access Control List (ACL) (Figure 12.15b)**
 - Created by decomposing the matrix by columns.
 - Each object maintains a list of users and their permitted access rights.
 - May also include default or public permissions.
2. **Capability Tickets (Figure 12.15c)**
 - Created by decomposing the matrix by rows.
 - Each user holds tickets specifying which objects and operations are allowed.
 - Tickets must be secure and unforgeable, usually managed by the OS.



Network Considerations

- When data is transmitted over a network, encryption may be required to protect sensitive data.
- Data access control is usually handled by database management systems (DBMS) or network database servers.

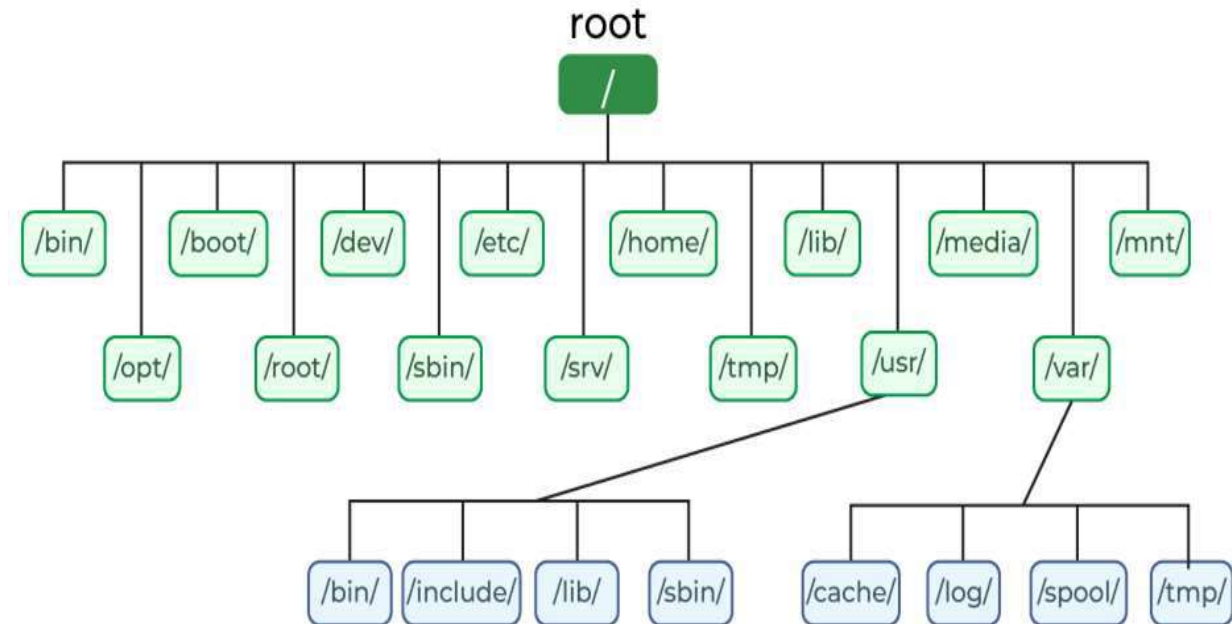
UNIX File System

In the UNIX file system, six types of files are distinguished:

1. **Regular, or ordinary:** A regular file is a normal file that stores data. This data can be created by a user, a software application, or a system program.
 - The file can contain any type of data such as text, images, programs, or documents.
 - The data is stored in blocks inside the storage device. The file system does not care about the format of the data inside the file. It simply treats the file as a continuous stream (sequence) of bytes.
Example: Files like .txt, .docx, .jpg, .pdf, .c are all regular files.
1. **Directory:** Contains a list of file names plus pointers to associated inodes (index nodes), described later. Directories are hierarchically organized (Fig shown).

Directory files are actually ordinary files with special write protection privileges so that only the file system can write into them, while read access is available to user programs.

The Unix file system is a hierarchical, tree-structured framework that organizes data into files and directories, starting from a single root directory (/).



UNIX File System

In the UNIX file system, six types of files are distinguished:

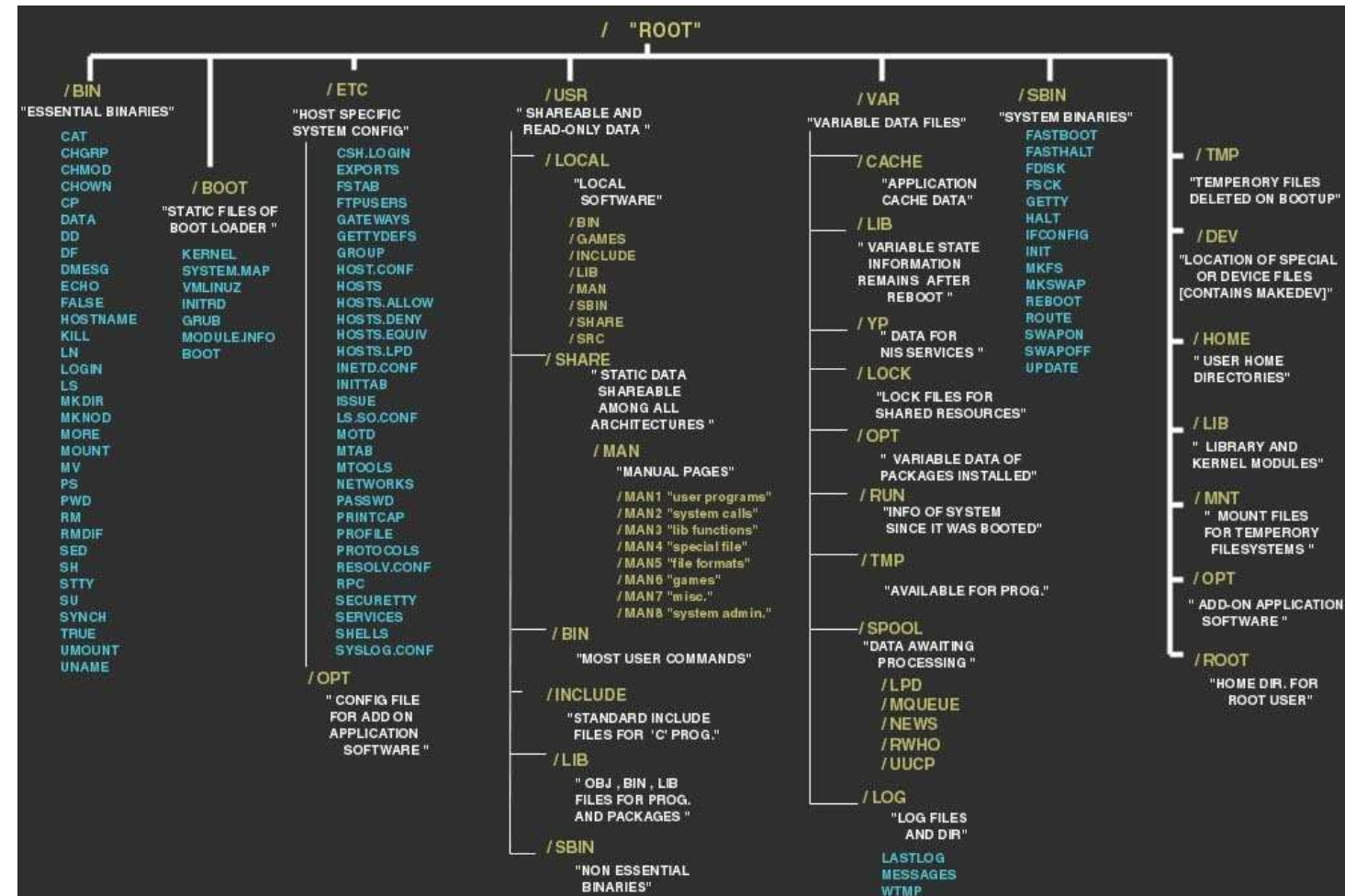
3. **Special:** Contains no data but provides a mechanism to map physical devices to file names.

The file names are used to access peripheral devices, such as terminals and printers. Each I/O device is associated with a special file.

4. **Named pipes:** a pipe is an interprocess communications facility. A pipe file buffers data received in its input so that a process that reads from the pipe's output receives the data on a first-in-first-out basis.

5. **Links:** In essence, a link is an alternative file name for an existing file.

6. **Symbolic links:** This is a data file that contains the name of the file it is linked to.



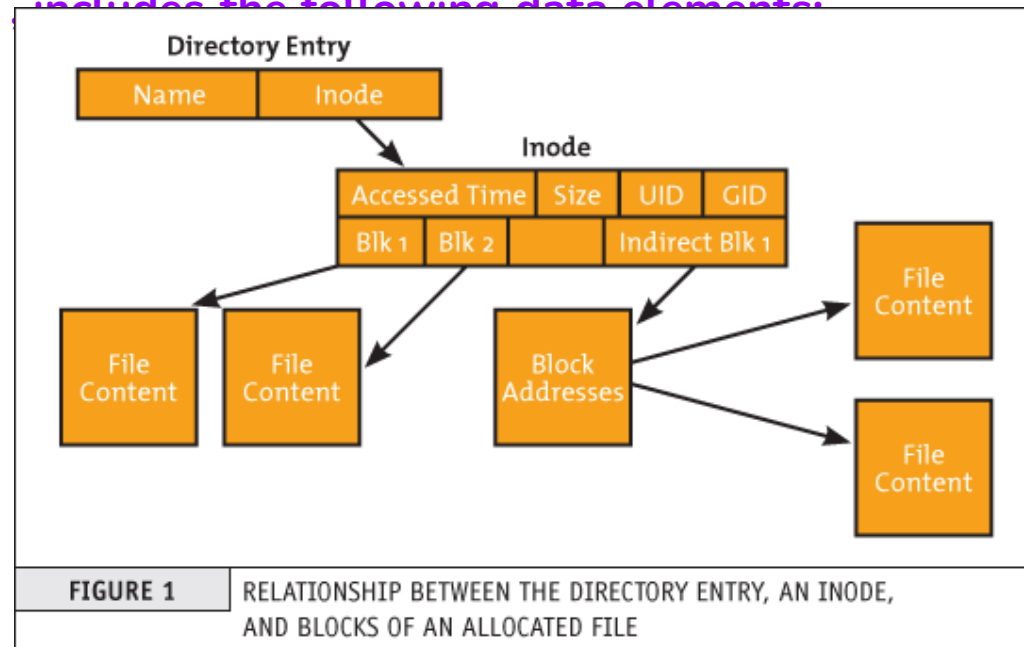
UNIX File System

In UNIX operating systems, every file is managed using a special data structure called an inode (index node).

What is an inode?

An inode is a data structure that stores important information about a file. It does not store the file name or the actual data, but it stores details about the file.

The attributes of the file as well as its permissions and other control information are stored in the inode. The exact inode structure varies from one UNIX implementation to another. The FreeBSD inode structure, shown in Figure 12.16, includes the following data elements:



here- FreeBSD is a free, open-source, Unix-like operating system derived from BSD (Berkeley Software Distribution).

Each file system in UNIX has at least one table that identifies the files in it. The entries in this table are called i-nodes (pronounced eye-nodes), and the indices of the i-nodes in this i-node table are called i-numbers. The i-nodes contain the file attributes and a map indicating where the blocks of the file are located on the disk.

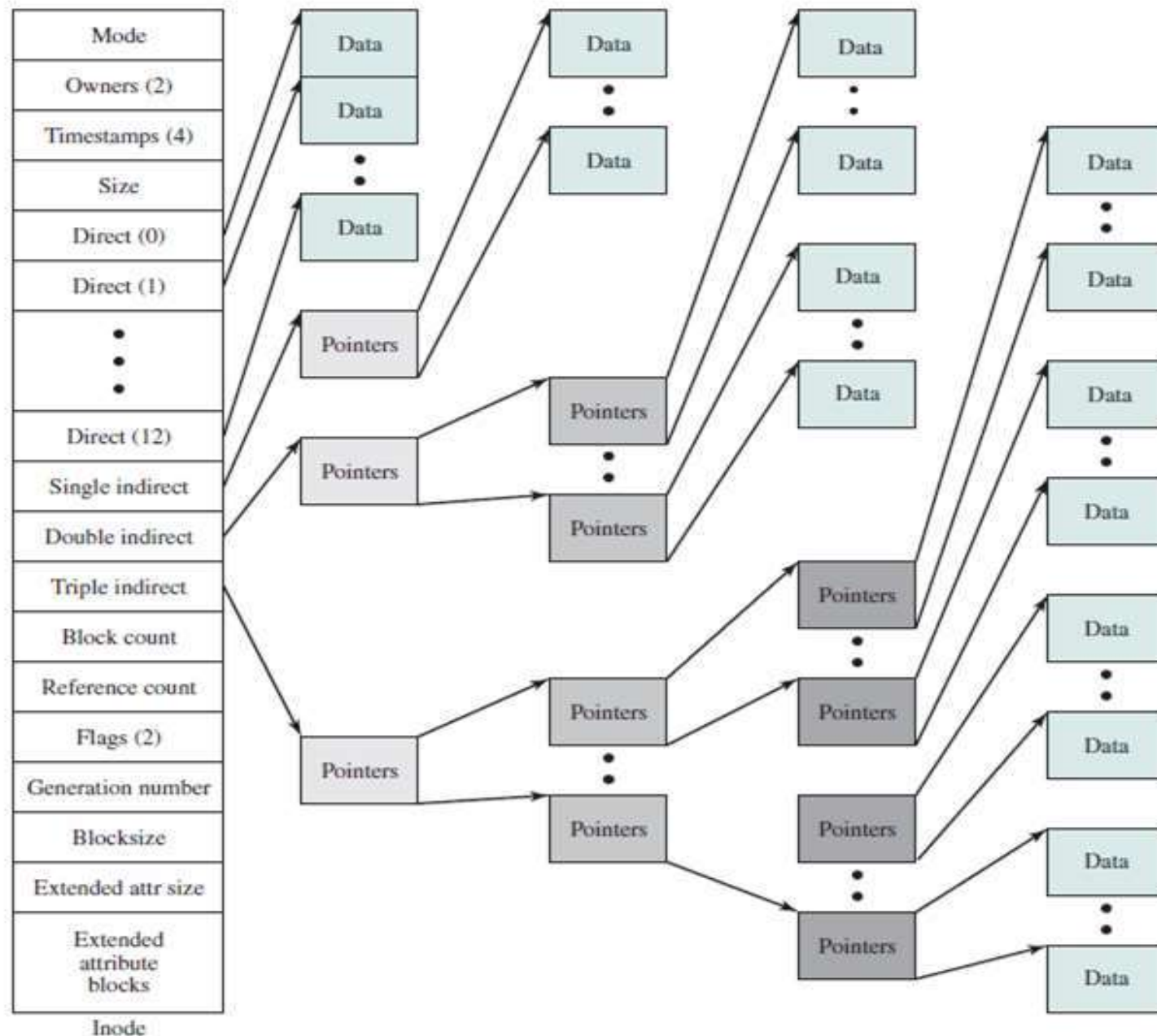


Figure 12.16 Structure of FreeBSD Inode and File

1. **File type and access mode**
 - Tells what type of file it is (regular file, directory, etc.).
 - Also stores permissions like read, write, and execute.
2. **Owner and group ID**-Identifies who owns the file (user) and which group it belongs to.
3. **File timestamps**- Stores different time information such as:
 - When the file was created
 - When it was last read
 - When it was last modified
 - When the inode was last updated

UNIX File System

4. **File size**-The total size of the file in bytes.
5. **Block pointers**-These pointers show where the file's actual data is stored on the disk.
6. **Number of disk blocks used**-Shows how many physical disk blocks are used to store the file data and related pointers.
7. **Number of directory references**-Indicates how many directory entries (file names) are linked to this inode.
8. **File flags**-Special system or user flags that describe file characteristics.
9. **Generation number**
 - A random number assigned when the inode is created.
 - Helps the system detect references to deleted files.
10. **Block size of the file**-Indicates the size of the data blocks used to store the file.
11. **The size of the extended attribute information**
12. **Zero or more extended attribute entries**

UNIX File System

File Allocation in UNIX -In UNIX systems, **files are stored on disk in blocks**. Allocation is dynamic, as needed, rather than using preallocation. Hence, the blocks of a file on disk are not necessarily contiguous.

An indexed method is used to keep track of each file, with part of the index stored in the inode for the file. In all UNIX implementations, the inode includes a number of direct pointers and three indirect pointers (single, double, triple).

1 **Block-based storage**

- Disk space is divided into **small units called blocks**.
- When a file is saved, it is stored in **one or more blocks**.

2 **Dynamic allocation**

- Blocks are **allocated only when needed**.
- The system **does not reserve all space in advance**.

3 **Non-contiguous storage**

- The blocks of a file **do not need to be next to each other on disk**.
- They can be **spread across different locations**.

To keep track of these scattered blocks, UNIX uses **indexing through the inode**.

UNIX File System

File Allocation in UNIX-Block Pointers in an Inode

In systems like **FreeBSD**, the inode contains **15 pointers (addresses)** that point to file data blocks.

1 Direct Pointers (First 12 pointers)

- The **first 12 addresses** point **directly to the first 12 data blocks of the file**.

Example

```
Inode → Block1
      → Block2
      → Block3
      ...
      → Block12
```

2 Single Indirect Pointer (13th pointer)

- Points to a **block that contains many addresses of data blocks**.

Structure: **Inode** → **Single Indirect Block** → **Data Blocks**

This allows the file to store **many more blocks**.

3 Double Indirect Pointer (14th pointer)

- Points to a block that contains **addresses of single indirect blocks**.

Structure: **Inode** → **Double Indirect Block** → **Single Indirect Blocks** → **Data Blocks**

This supports **even larger files**.

4 Triple Indirect Pointer (15th pointer)

- Adds **one more level of indexing**.

Structure: **Inode** → **Triple Indirect Block** → **Double Indirect** → **Single Indirect** → **Data Blocks**

This allows **very large files to be stored**.

Block Capacity Example (FreeBSD)

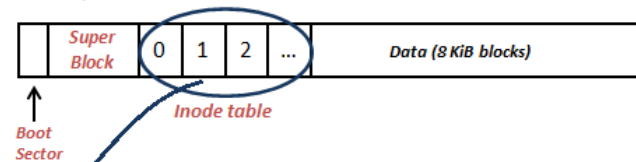
- Minimum block size = **4 KB**
- Each block can store **512 addresses**
Because of multiple levels of indexing, a file can grow to **more than 500 GB**.

UNIX File System

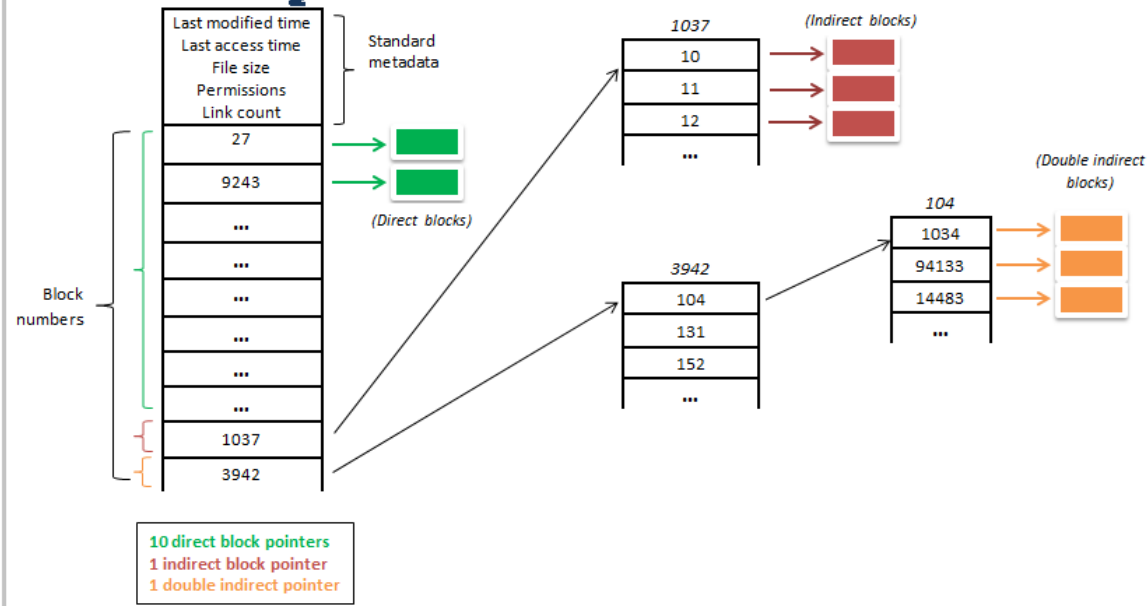
File Allocation in UNIX-Block Pointers in an Inode

In systems like **FreeBSD**, the inode contains **15 pointers (addresses)** that point to file data blocks.

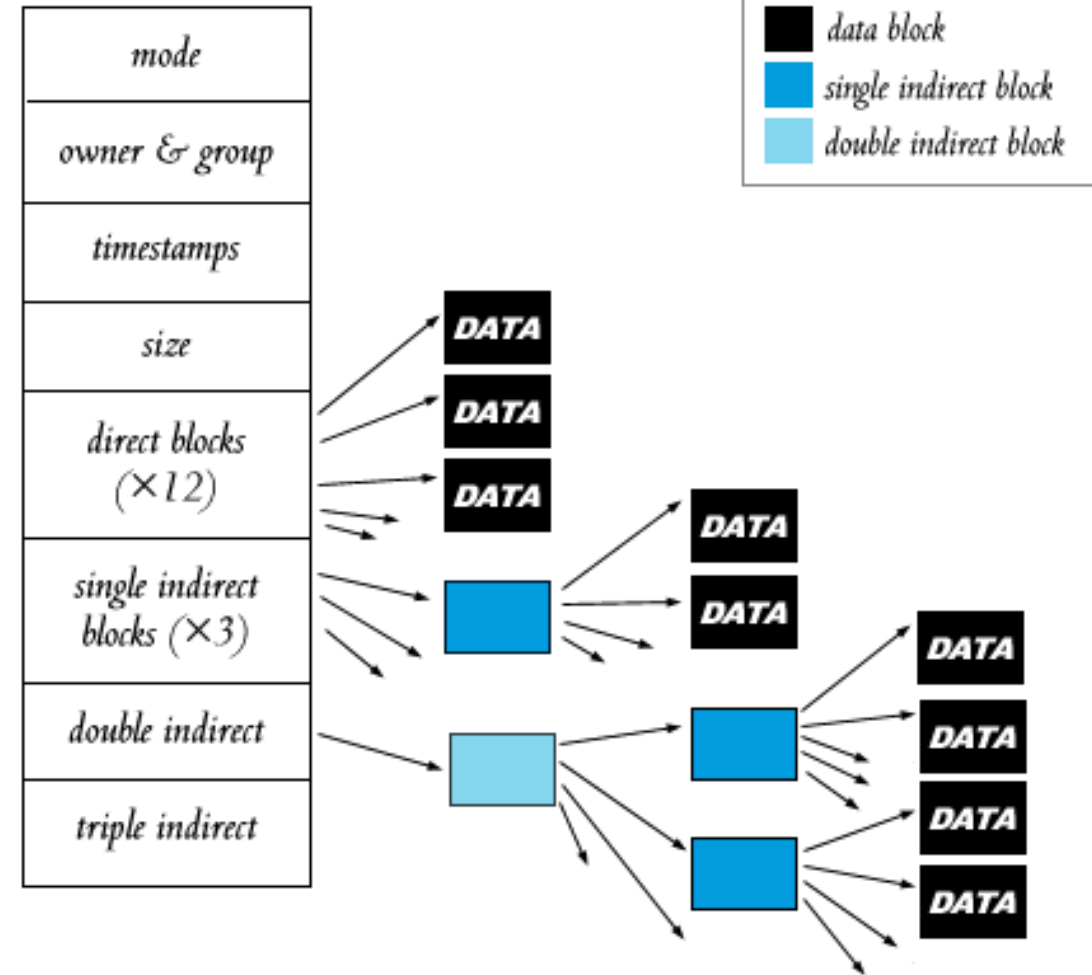
Unix File System



Inode entry



inode



UNIX File System

In a UNIX system (like **FreeBSD**), an **i-node contains pointers that show where the file data blocks are stored on the disk.**

With a **4 KB block size**, the file can grow up to **more than 500 GB** using different levels of pointers.

Level	Number of Blocks	Maximum Data Size
Direct	12 blocks	48 KB
Single Indirect	512 blocks	2 MB
Double Indirect	512 × 512 blocks	1 GB
Triple Indirect	512 × 512 × 512 blocks	512 GB

Table 12.4 Capacity of a FreeBSD File with 4-Kbyte Block Size

Advantages of This Scheme

- 1 ☐ **Small inode size**-The inode has a **fixed and small size**, so it can stay in **main memory for faster access**.
- 2 ☐ **Fast access for small files**-Small files use **direct pointers**, so they require **little or no extra lookup**, which speeds up access.
- 3 ☐ **Supports very large files**-Using multiple levels of indirect blocks allows files to grow **very large (hundreds of GB)**.

UNIX File System

Directories in UNIX -

In UNIX systems, **directories are organized in a hierarchical tree structure.**

Hierarchical Tree Structure

- The **top directory** is called the **root directory**.
- A directory can contain:
 - **Files**
 - **Other directories (subdirectories)**

A **subdirectory** is simply a directory **inside another directory**.

Root
Example structure:

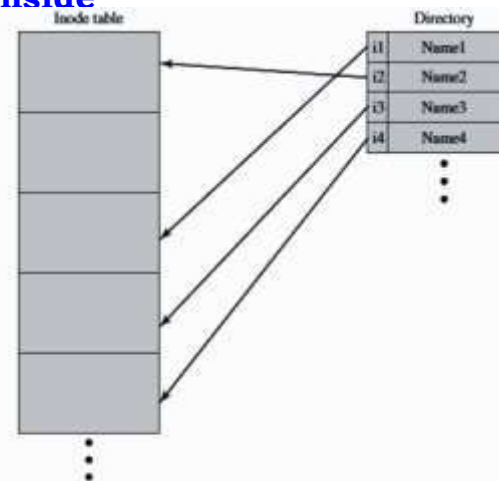
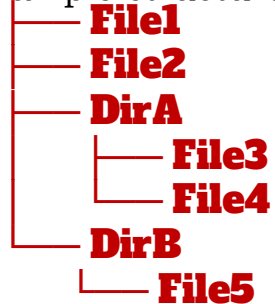


Figure 12.17 UNIX Directories and Inodes

Directory as a File

In UNIX, a **directory is actually a special type of file.**

It contains: (see fig on next slide)

- **File names**
- **Pointers to their inodes**

Each entry in a directory is called a **directory entry (dentry)**. Each **dentry** contains:

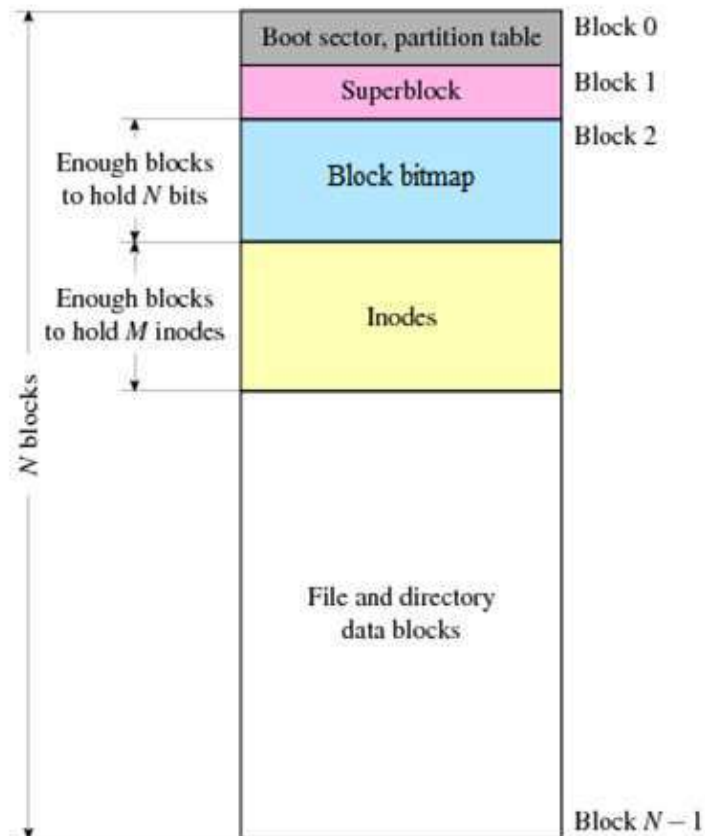
1. **File or directory name**
2. **i-number (inode number)**

The **i-number** is used to find the **inode of that file** in the inode table.

Example: The system uses **3456** to locate the **inode** of **report.txt**
 Directory Entry
 File name → **report.txt**
 i-number → **3456**

UNIX File System

Figure 8
BSD File System Disk Partition

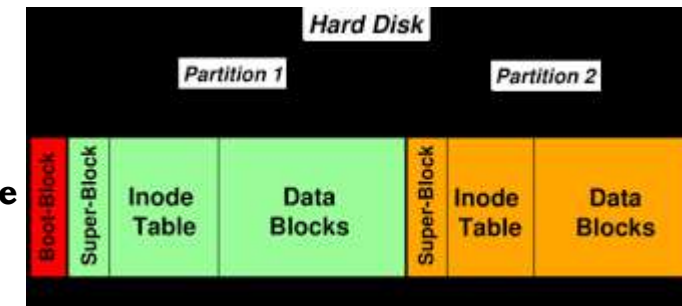


Volume Structure in UNIX

A UNIX file system is stored on a **logical disk or disk partition**.

It is divided into several parts:

- 1 **Boot Block**- Contains the **program needed to start (boot) the operating system**.
- 2 **Superblock**- Stores **important information about the file system**, such as:
 - Size of the partition
 - Size of the inode table
 - File system configuration
- 3 **Inode Table**- Contains **all the inodes of files and directories** in the system.
- 4 **Data Blocks**- These blocks store the **actual data of files and directories**.



UNIX File System

Traditional UNIX File Access Control -UNIX systems control **who can access a file and what they can do with it** using a **file access control mechanism**.

1 Users and Groups

- Every user in UNIX has a **User ID (UID)**.
- Each user belongs to a **primary group** and may belong to **other groups**.
- Groups are identified using a **Group ID (GID)**.

When a **file is created**:

- It is owned by the **user who created it**.
- It also belongs to a **group**.

These details (**UID, GID, permissions**) are stored in the **inode** of the file.

2 File Permission Bits

Each file has **12 protection bits. 9 Permission Bits (Basic Permissions)**

These specify **Read (r), Write (w), Execute (x)** permissions for:

Category	Meaning
Owner	Person who created the file
Group	Members of the file's group
Others	All other users

example : rw-r-----

explanation -Owner: Read + Write

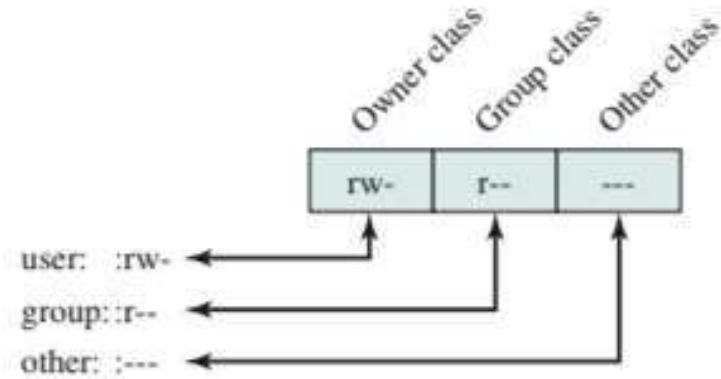
Group: Read only

Others: No access

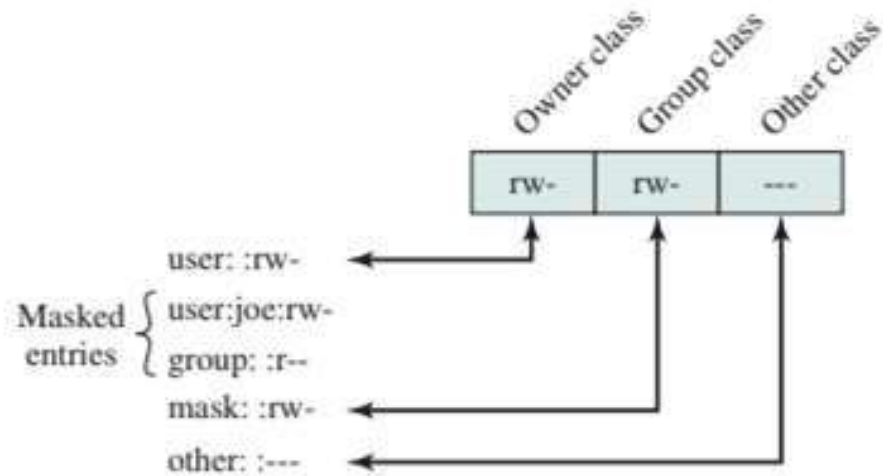
Permissions for Directories

Permission	Meaning
Read	List files in directory
Write	Create, rename, or delete files
Execute	Search inside directory

UNIX File System



(a) Traditional UNIX approach (minimal access control list)



(b) Extended access control list

Figure 12.18 UNIX File Access Control

ls -l

rwX Execute let's you execute executables
 read write execute

4 2 1 4 2 1 4 2 1
- rwx rwx rwx
 user group other
 7 7 7

Permissions (mode)	Number
rwx	7 (4+2+1)
rw-	6 (4+2)
r-x	5 (4+1)
r--	4
-wx	3 (2+1)
-w-	2
--x	1
---	0

chown (change ownership) - ex chown root file1

chmod (change mode [permissions]) ex chmod 540 file1

Example

permissions owner group other

-rwxrw-r-- 1 jsmith student 365 Feb 22 15:31 lab1.c

links owner group file-size creation date filename

UNIX File System

3 Special Permission Bits

The remaining **3 bits** provide special behavior.

(a) SetUID (Set User ID)

- When a program is executed, it **runs with the permissions of the file owner**, not the user running it.
- Used for **privileged programs**.
Example: system password programs.

(b) SetGID (Set Group ID)

For **files**: -Program runs with **group privileges of the file**.

For **directories**: New files created inside **inherit the directory's group**.

(c) Sticky Bit

For **directories**: -Only the **file owner can delete or rename the file**, even if others have write permission.

Example: Shared directories like **/tmp**.

4 Superuser (Root)

UNIX has a special user called **superuser (root)**.

- Has **full access to all files**.
- Not restricted by normal file permissions.
Programs owned by root with **SetUID** can give **very powerful access**, so they must be written carefully.

5 Limitation of Traditional UNIX Permissions

The system works well for **simple access control**, but it has limitations.

Example problem:

- File X → access for **Users A and B**
- File Y → access for **Users B and C**

This requires **multiple groups**, and managing many groups becomes difficult.

To solve this problem, modern UNIX systems use **Access Control Lists (ACLs)**.

6 Protection Domains

In UNIX: A **protection domain** is associated with a **user**.

- When the system changes the **user ID temporarily** (like with SetUID), the **domain**

LINUX VIRTUAL FILE SYSTEM

- Linux includes a versatile and powerful file-handling facility, designed to support a wide variety of file management systems and file structures.
- Linux supports **many types of file systems** (like EXT4, FAT, NTFS, JFS, etc.) Instead of making programs deal with each one differently, Linux uses something called: **Virtual File System (VFS)**, VFS acts like a **common interface (middle layer)** between user programs and actual file systems.

Imagine:

- **One USB uses FAT (Ms DOS)**
- **Hard disk uses EXT4 (Linux)**
- **Another partition uses NTFS (windows)**

Every program must understand all file systems so with VFS: Programs use ONE standard way to access files that is “One interface for all file systems”

Basic properties of a file in Linux Every file in Linux has:

- **Name** (symbolic name)
- **Owner**
- **Permissions** (who can read/write)
- **Location in directory**
- **Data** And operations allowed: Create, Read, Write, Delete

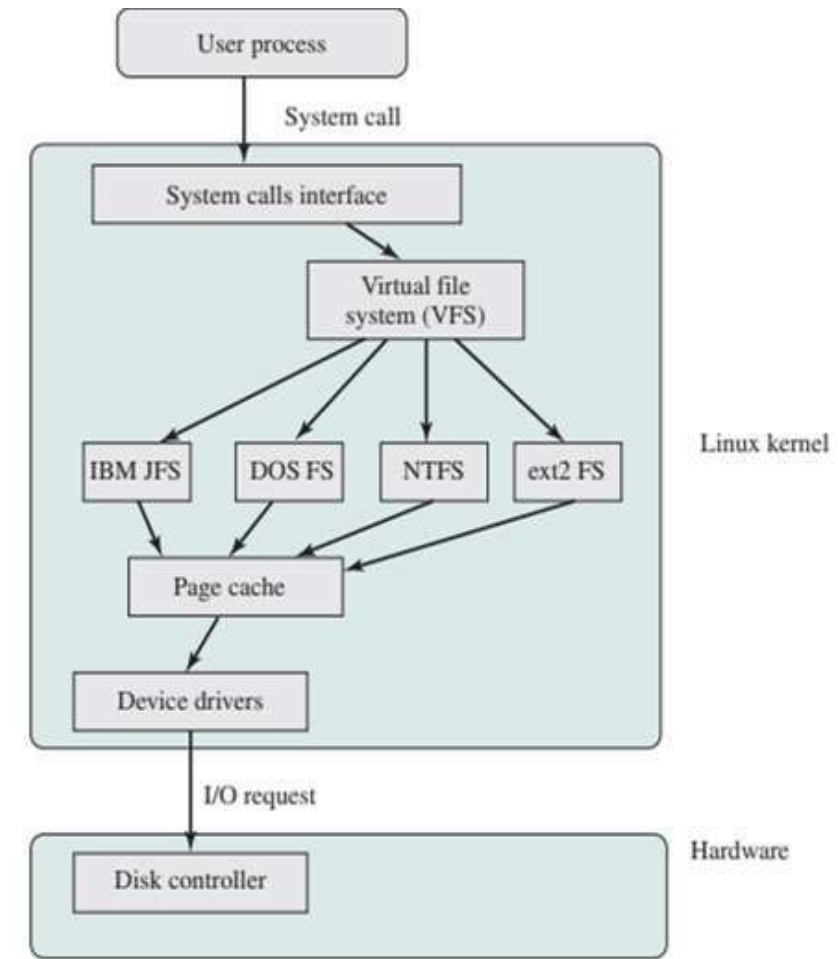


Figure 12.19 Linux Virtual File System Context

LINUX VIRTUAL FILE SYSTEM

How does VFS work? (Step-by-step flow)

Step 1: User makes request Example: read(file)

Step 2: VFS receives request

- VFS does **common processing**
- It doesn't care which file system is used

Step 3: Mapping function

- VFS passes request to a **mapping layer**
- This layer converts request into format understood by specific file system
Example:
- VFS → EXT4 format
- VFS → FAT format

Step 4: Actual file system works

- The real file system (EXT4, FAT, JFS, etc.)
- Performs the operation on disk

Step 5: Device driver -Converts operation into **hardware-level instructions**

Step 6: Result returns back-Goes back through same path to user

User Program



System Call (read/write)



VFS (common interface)



Mapping Function



Actual File System (EXT4/FAT/etc.)



Device Driver



Disk

Advantages of VFS-

1. Supports **multiple file systems**
2. Provides **uniform interface**
3. Hides complexity from users
4. Makes Linux very flexible

LINUX VIRTUAL FILE SYSTEM

How does VFS work? (Step-by-step flow)

Even though Linux is written in **C**, VFS behaves like OOP.

How?

- Uses **structures (structs)** instead of classes
- Each object contains:
 - Data
 - Function pointers

Example objects:

- inode (file info)
- file object
- superblock

These act like **objects + methods**

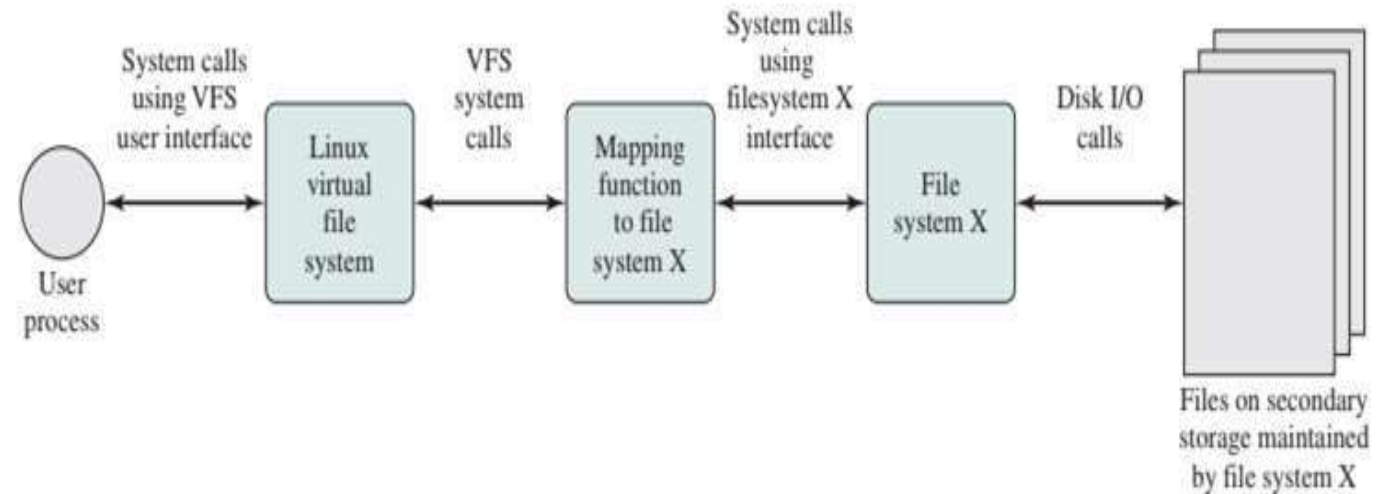


Figure 12.20 Linux Virtual File System Concept

The four primary object types in VFS are as follows:

- **Superblock object**: Represents a specific mounted file system
- **Inode object**: Represents a specific file
- **Dentry object**: Represents a specific directory entry
- **File object**: Represents an open file associated with a process

LINUX VIRTUAL FILE SYSTEM

1. Superblock Object in File System

Definition:

The **superblock** is a key data structure that stores metadata describing a file system. It resides in a special disk sector and is loaded into memory when the file system is mounted.

Acts as the control structure of a file system, enabling efficient management and access of files.

Key Information Stored:

- Device on which file system is mounted
- File system type
- Dirty flag (flag, to indicate that the superblock has been changed but not written back to disk)
- Read-only and other status flags
- Pointer to root directory
- List of open files
- Semaphore for access control
- Basic block size of file system

Superblock Operations-The operations object defines the object methods (functions) that the kernel can invoke against the superblock object:

- **read_inode** – Read inode from disk
- **write_inode** – Write inode to disk
- **put_inode** – Release inode
- **delete_inode** – Delete inode
- **notify_change** – Update inode attributes
- **put_super** – Release superblock on unmount
- **write_super** – Save superblock to disk
- **statfs** – Get file system statistics
- **remount_fs** – Remount with new options
- **clear_inode** – Clear inode data

LINUX VIRTUAL FILE SYSTEM

2. Inode Object in File System

Definition:

An *inode* is a data structure associated with each file that stores all metadata **except the file name and actual data**.

- File name is stored in directory entries (**dentry**), not in inode
- Inode acts as the **metadata manager** of a file, Provides essential information required by the OS to manage, locate, and control access to files.

Information Stored in Inode:

- File owner and group
- File permissions
- Access, modification, and creation times
- File size
- Number of links (hard links)

The methods defined for the inode object include the following:

- **create** : Creates a new inode for a regular file associated with a dentry object in some directory.
- **lookup** : Searches a directory for an inode corresponding to a file name
- **mkdir** : Creates a new inode for a directory associated with a dentry object in some directory.

LINUX VIRTUAL FILE SYSTEM

3. The Dentry Object

A dentry (directory entry) is a specific component in a path. The component may be either a directory name or a file name. Dentry objects facilitate access to files and directories and are used in a dentry cache for that purpose. The dentry object includes a pointer to the inode and superblock. It also includes a pointer to the parent dentry and pointers to any subordinate dentries.

4. The File Object

Definition:

A *file object* represents an **open file instance** in memory. It is created when a process calls `open()` and destroyed when `close()` is invoked.1. **The file object consists of a number of items, including the following:**

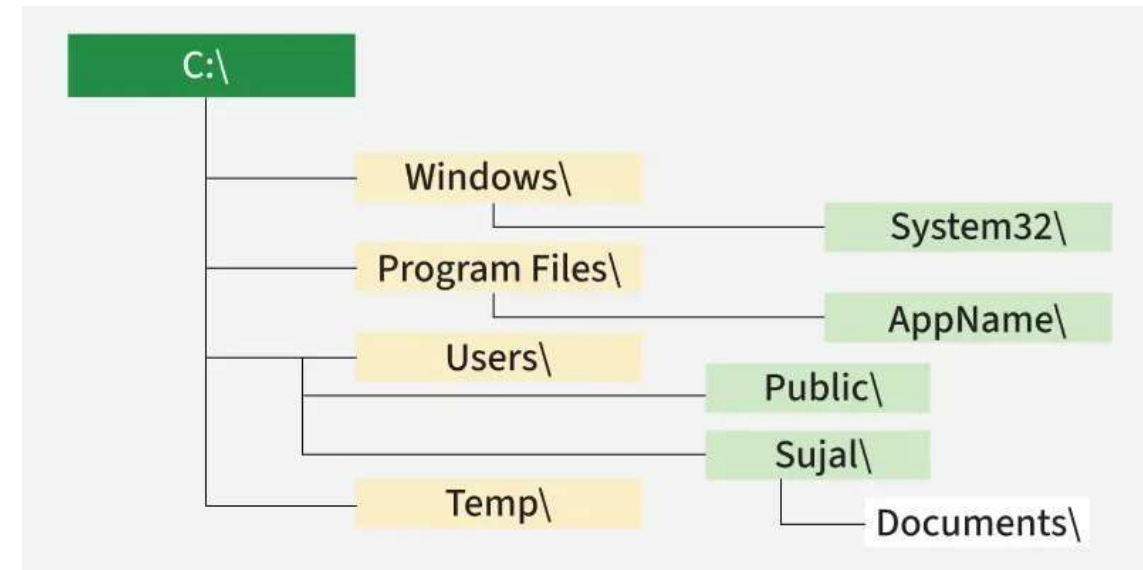
- **Dentry object associated with the file**
- **File system containing the file**
- **File objects usage counter**
- **User's user ID**
- **User's group ID**
- **File pointer**, which is the current position in the file from which the next operation will take place The file object also includes an inode operations object that describes the file system's implemented functions that the VFS can invoke on a file object. The methods defined for the file object include read, write, open, release, and lock.

WINDOWS FILE SYSTEM

The developers of Windows NT designed a new file system, the New Technology File System (NTFS), which is intended to meet high-end requirements for workstations and servers.

Examples of high-end applications include the following:

- Client/server applications such as file servers, compute servers, and database servers
 - Resource-intensive engineering and scientific applications
 - Network applications for large corporate systems
- ex- web browsers, email clients, messaging apps, video conferencing tools, file transfer tools, and online gaming.



WINDOWS FILE SYSTEM : Key Features of NTFS

1. Recoverability (Crash Recovery)

- NTFS can **recover data after system crashes or disk failures.**
- It uses a **transaction system:**
 - Every change is treated as a complete action (**all or nothing**).
 - If a crash happens, changes are either:
 - **Completed**, or
 - **Rolled back (undone)**
- Also stores **backup copies of critical data**, so structure is not lost.
- *Think of it like saving progress in a game—either the level is saved fully or not at all.*

2. Security

- Each file has a **security descriptor** (like permissions).
- Controls: Who can **read/write/modify** the file Based on **Windows security model**.

Example: Only authorized users can access certain files.

WINDOWS FILE SYSTEM

3. Supports Large Files and Disks

- NTFS can handle:
 - Very large files
 - Huge storage disks
- More efficient than older systems like FAT.

Suitable for modern systems with TB-level storage.

5. Journaling

- NTFS keeps a **log (journal)** of all file changes.
- Helps:
 - Track modifications
 - Recover quickly after crashes

Like a history log of everything happening in the file system.

4. Multiple Data Streams

- A single file can store **multiple streams of data.**
- Example:
 - One stream = file content
 - Another stream = extra information (metadata)

Useful for compatibility with systems like Mac (data + resource fork).

WINDOWS FILE SYSTEM

6. Compression and Encryption

- **Files/folders can be:**
 - **Compressed** → saves space
 - **Encrypted** → increases security
- Done automatically (transparent to user).

Example: Sensitive files can be encrypted to prevent unauthorized access.

7. Hard Links and Symbolic Links

- **Hard Link:** Same file accessed using **different names (same disk)**
- **Symbolic Link:** Shortcut pointing to a file (can be on **different disks**)
- **Mount Points:** A disk can appear as a folder instead of drive letter.

Similar to shortcuts but more powerful.

WINDOWS FILE SYSTEM - NTFS Volume and File Structure

Basic Disk Storage Concepts

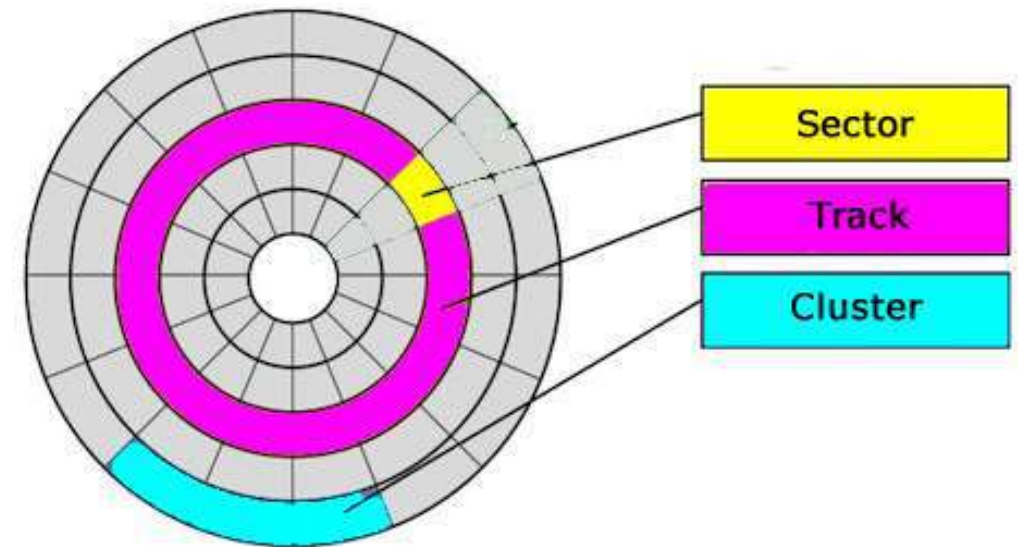
1. Sector

- Smallest **physical storage unit** on addressable unit of data storage on a hard disk drive (HDD) or solid-state drive (SSD).
- A sector is the smallest physical, , typically holding 512 bytes (or power of 2) or 4KB (4096 bytes) of data. Sectors are organized into tracks on disks and form the foundation for file systems to allocate space for files

2. Cluster

- A group of **one or more contiguous sectors**
- NTFS allocates space in terms of clusters

Cluster = allocation unit (file gets space in clusters, not sectors).



•What are tracks and sectors?



WINDOWS FILE SYSTEM - NTFS Volume and File Structure

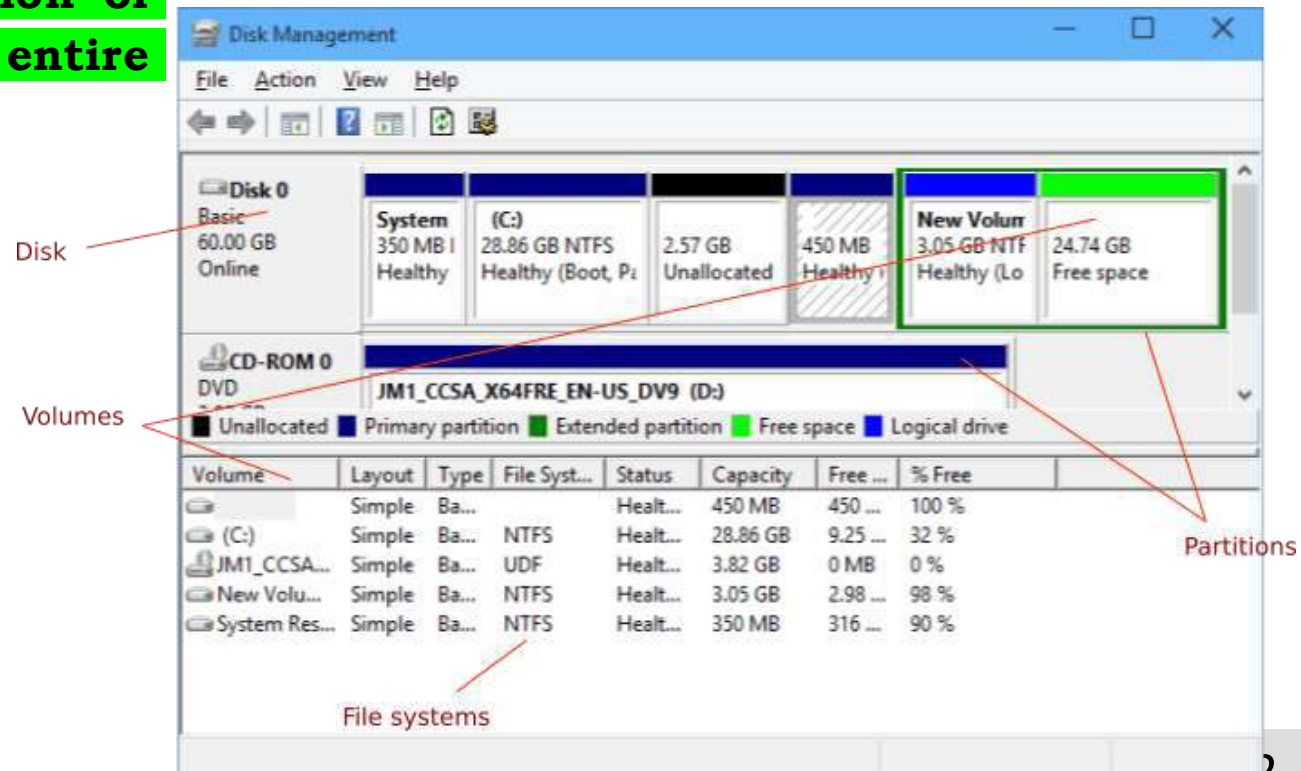
Basic Disk Storage Concepts

3. Volume

- A volume in a file system is a single, accessible storage area with a dedicated file system (like NTFS, FAT32, or ext4), **representing a logical division of physical storage, such as a partition or an entire disk.**
- Made up of multiple clusters
- Contains:
 - File system information
 - Files & folders
 - Free (unallocated) space

A volume can be:

- Entire disk
- Part of a disk
- Spread across multiple disks



WINDOWS FILE SYSTEM - NTFS Volume and File Structure

Layout of NTFS Volume

1. Boot Sector-

- Located at the **beginning of the volume**
- Contains:
 - Volume information
 - File system structure details
 - Boot program (startup code)

Used when system starts (booting process).

2. Master File Table (MFT) –

- Core of NTFS (heart of file system)
- Stores **information about all files and directories**
- Organized like a **table (rows = records)**

Each file = one row in MFT

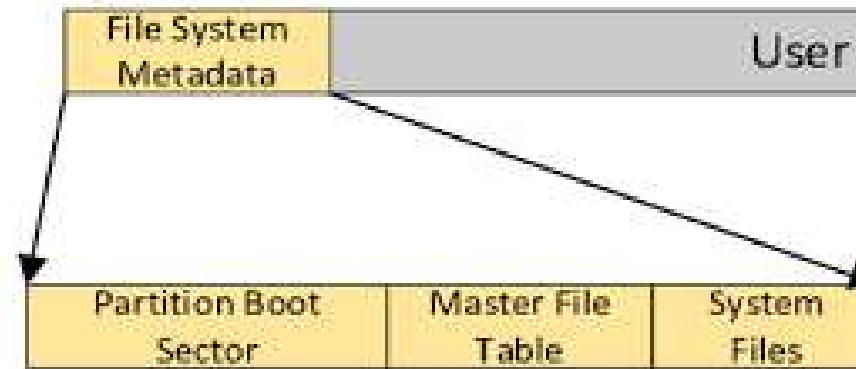
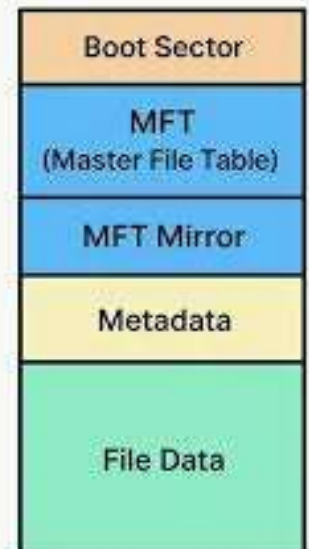


Figure -layout of the NTFS volume.

STRUCTURE OF THE NTFS FILE SYSTEM



WINDOWS FILE SYSTEM - NTFS Volume and File Structure

3. System Files Region

After MFT, NTFS stores important system files:

- **MFT Mirror (MFT2)**

- Backup of first few MFT records
- Helps in **recovery if MFT is damaged**

- **Log File (Journal)**

- Stores **transaction records**
- Used for **crash recovery**

- **Cluster Bitmap**

- Keeps track of:
 - Used clusters
 - Free clusters
 - *Helps in space management*

- **Attribute Definition Table**

- Types of attributes allowed
- Which can be indexed or recovered

◆ Master File Table (MFT)

- Organized as **records of 1024 bytes each**
- Each record represents:
 - A **file or directory**
 - Even the **MFT itself is treated as a file**

File Storage in MFT

Case 1: Small File-Entire file stored **inside MFT record**, *Faster access*

Case 2: Large File

- MFT record stores:
 - File metadata
 - Pointers to actual data blocks (clusters)
 - *Actual content stored outside MFT*

◆ Attributes in MFT

Each file is described using **attributes**, such as:

- File name
- Size
- Security info
- Data location

WINDOWS FILE SYSTEM - NTFS Volume and File Structure

Table 12.6 Windows NTFS File and Directory Attribute Types

Attribute Type	Description
Standard information	Includes access attributes (read-only, read/write, etc.); time stamps, including when the file was created or last modified; and how many directories point to the file (link count)
Attribute list	A list of attributes that make up the file and the file reference of the MFT file record in which each attribute is located. Used when all attributes do not fit into a single MFT file record
File name	A file or directory must have one or more names.
Security descriptor	Specifies who owns the file and who can access it
Data	The contents of the file. A file has one default unnamed data attribute and may have one or more named data attributes.
Index root	Used to implement folders
Index allocation	Used to implement folders
Volume information	Includes volume-related information, such as the version and name of the volume
Bitmap	Provides a map representing records in use on the MFT or folder

Note: Green-colored rows refer to required file attributes; the other attributes are optional.

WINDOWS FILE SYSTEM - NTFS Volume and File Structure

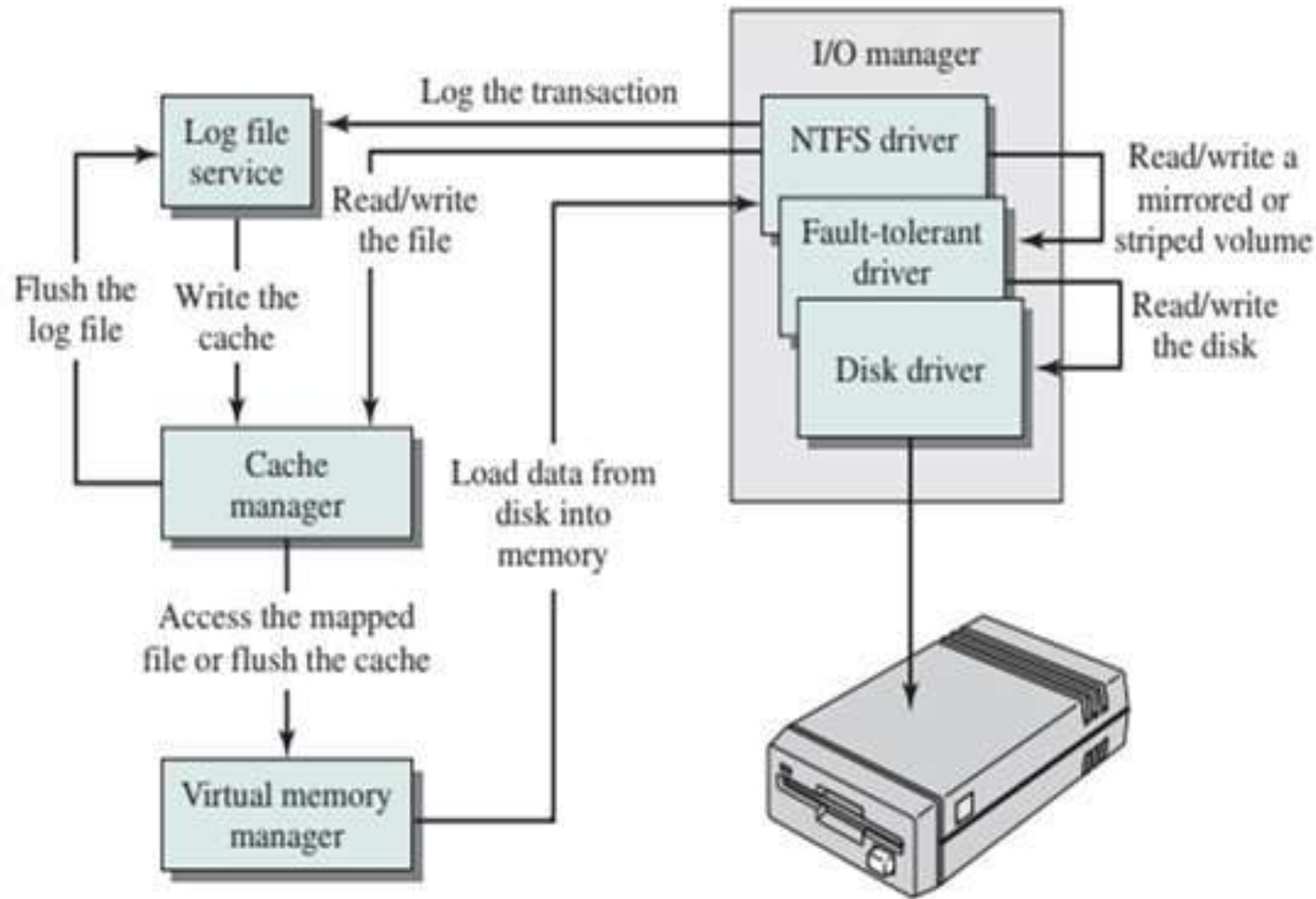


Figure 12.22 Windows NTFS Components

Recoverability

NTFS makes it possible to recover the file system to a consistent state following a system crash or disk failure. The key elements that support recoverability are as follows (Figure 12.22):

WINDOWS FILE SYSTEM - NTFS Volume and File Structure

Recoverability

The key elements that support recoverability are as follows (Figure 12.22):

- **I/O manager:** Includes the NTFS driver, which handles the basic open, close, read, and write functions of NTFS. In addition, the software RAID module FTDISK can be configured for use.
- **Log file service:** Maintains a log of file system metadata changes on disk. The log file is used to recover an NTFS-formatted volume in the case of a system failure (i.e., without having to run the file system check utility).
- **Cache manager:** Responsible for caching file reads and writes to enhance performance. The cache manager optimizes disk I/O.
- **Virtual memory manager:** The NTFS accesses cached files by mapping file references to virtual memory references and reading and writing virtual memory.

WINDOWS FILE SYSTEM - NTFS Volume and File Structure

Recoverability

- In general terms, these are the steps taken to ensure recoverability, as described in [RUSS11]:

1. **NTFS** first calls the **log file system to record in the log file** (in the cache) any transactions that will modify the volume structure.

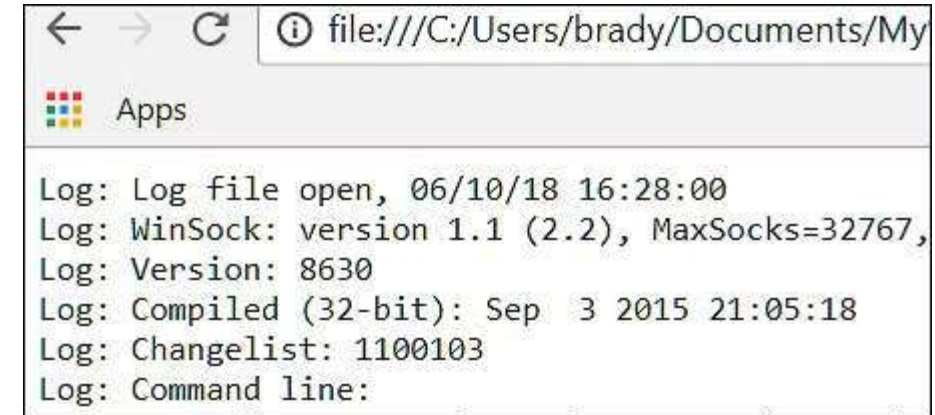
Log files are essential records of system activity, offering a detailed, chronological history of operations, errors, and user actions, which are crucial for troubleshooting, security auditing, and performance optimization. They enable administrators to identify bottlenecks, diagnose application crashes, detect unauthorized access, and ensure high system availability

WINDOWS FILE SYSTEM - NTFS Volume and File Structure

Recoverability

- In general terms, these are the steps taken to ensure recoverability, as described in [RUSS11]:

2. NTFS **modifies the volume (in the cache).**
3. The **cache manager calls** the log file system to prompt it to flush the log file to disk.
4. Once the **log file updates are safely on disk**, the cache manager flushes the volume changes to disk.



WINDOWS FILE SYSTEM - NTFS Volume and File Structure

Unix/Linux Log Systems

Unix and Linux systems follow a centralized, text-based logging approach where logs are generally stored in the `/var/log` directory.

- Key Log Locations (`/var/log/`):
 - `/var/log/syslog` (or `/var/log/messages`): General system activity logs.
 - `/var/log/auth.log` (or `secure`): Authentication logs (sudo, SSH logins).
 - `/var/log/kern.log`: Kernel-related messages.
 - `/var/log/dmesg`: Hardware and boot-time messages.
- Logging Daemon: `syslogd` (or `rsyslogd`) manages logging, while modern systems often use `systemd-journald` to store logs in a binary format (viewable via `journalctl`).
- Commands: `cat`, `tail -f`, `grep`, `less` are used to monitor logs in real-time

Windows Log Systems

Windows uses a structured, binary log system designed to be viewed primarily through the Event Viewer or specialized APIs, rather than text editors.

[YouTube](#) +4

- Event Viewer (`eventvwr.msc`): Categorizes logs into Application, Security, and System.
 - System Log: Records events from Windows system components (drivers, service failures).
 - Application Log: Records application-level logs.
 - Security Log: Records security events (logins, resource access).
- Log Storage: Many logs are kept in `C:\Windows\System32\winevt\Logs\`.
- Commands: `Get-EventLog` Or `Get-Content -Wait` in PowerShell are used for monitoring